Models for Programming Paradigms

1. Foundations & Preliminaries **Key Concepts:** Syntax vs Semantics vs Pragmatics. **Methods:** Operational Semantics (SOS), Denotational, Axiomatic. **Properties:** Termination, Determinacy, Compositionality.
2. Math & Logic **Induction:** Mathematical, Structural, Well-Founded, and Rule Induction. **Domain Theory:** Partial Orders (PO), CPO, Monotone and Continuous Functions. **Fixpoint:** Kleene's Theorem ($\text{fix}(f) = \bigsqcup f^n(\bot)$) and the Immediate Consequence Operator (ICO).
3. Imp Semantics **IMP Language:** Syntax and States (Σ). **Operational Semantics:** Inference rules for `Aexp`, `Bexp`, `Com`. **Denotational Semantics:** Functions $\mathcal{C}[\![c]\!]$, handling *Lifting*, and `while` denotation. **Axiomatic Semantics:** Hoare Logic rules. **Equivalence:** Consistency Theorem between OS and DS.
4. HOFL **Higher-Order Functional Language:** Syntax, Types, and Inference Rules. **Evaluation:** Canonical Forms, Lazy (Call-by-Name) vs Eager (Call-by-Value). **Semantic Domains:** Continuous function spaces, semantics of λ-abstractions and recursion. **Haskell:** Conceptual mapping (lists, lazy evaluation).
5. Concurrency **Paradigms:** Message Passing (Erlang) vs Shared Memory/Channels (Go). **CCS:** Syntax, LTS, and transition rules (τ). **Bisimulation:** Definition of Strong Bisimulation, Attacker/Defender game. **Logic:** HML (Hennessy-Milner Logic) and Characterization Theorem. **Case Studies:** Modeling Buffers and Mutual Exclusion (Peterson).
6. Real Languages **Haskell:** Functional patterns (Guards, Data types), Type Classes, Lazy Evaluation. **Erlang:** Actor Model, Asynchronous Message Passing, `receive` with timeout. **Go:** Goroutines, Buffered vs Unbuffered Channels, `select` non-determinism.
7. Advanced Concurrency **Weak Bisimulation:** Abstraction from internal actions (τ), Weak Transition ($\alpha\Longrightarrow$). **Congruence:** Observational Congruence and Milner's τ-laws. **Temporal Logics:** Linear Time (LTL) vs Branching Time (CTL). **Mu-Calculus:** Syntax and Fixpoints for Safety (ν) and Liveness (μ) properties.

Practice & Review

1. Exam Questions A comprehensive list of past exam questions mapped to specific answers.
2. Exercises Collection of solved exercises on Domains, IMP, HOFL, CCS, and Real Languages.

Foundations & Preliminaries

This module establishes the rigorous mathematical framework necessary to define, analyze, and compare the semantics of programming languages.

1. Structure and Meaning of a Language

A programming language is defined by four essential components:

1. **Syntax:** Defines the sequences of symbols that constitute well-formed programs. Formalized via regular expressions, context-free grammars (CFG), or BNF notation.
2. **Types:** Restrict the syntax to enforce safety properties and prevent runtime errors. Type systems are defined via logical inference rules.
3. **Pragmatics:** Guidelines on the effective use of the language, best practices, and common patterns.
4. **Semantics:** Assigns an unambiguous meaning to well-typed programs, providing a formal model for programmers and implementers.

---

2. Methods of Formal Semantics

There are three main approaches to formalizing the meaning of programs.

2.1 Operational Semantics (OS)

Describes meaning in terms of execution on an abstract machine. It is based on the **SOS (Structural Operational Semantics)** formalism by Gordon Plotkin.

- **Small-step (SOS):** Describes the step-by-step evolution ($e_0 \to e_1 \to \ldots$). Fundamental for modeling concurrency and non-terminating systems.
- **Big-step (Natural Semantics):** Directly relates the initial state to the final result, hiding intermediate steps.
  - *Judgment for Expressions:* $\langle a, \sigma \rangle \to n$ (expression a in state σ evaluates to n).
  - *Judgment for Commands:* $\langle c, \sigma \rangle \to \sigma'$ (command c in state σ converges to state σ').

  Note: Big-step semantics does not produce a derivation tree for programs that diverge (do not terminate).

2.2 Denotational Semantics (DS)

More abstract than OS, it models programs as **mathematical functions** over specific domains, ignoring execution details.

Compositionality Principle: The meaning of a compound construct depends *only* on the meanings of its constituents:

$$\mathcal{M}[\![OP(c_1, c_2)]\!] = F(\mathcal{M}[\![c_1]\!], \mathcal{M}[\![c_2]\!])$$

Interpretation functions for an imperative language (IMP):

- **Aexp:** $\mathcal{A}[\![\cdot]\!] : \text{Aexp} \to (\Sigma \to \mathbb{Z})$
- **Bexp:** $\mathcal{B}[\![\cdot]\!] : \text{Bexp} \to (\Sigma \to \mathbb{B})$
- **Com:** $\mathcal{C}[\![\cdot]\!] : \text{Com} \to (\Sigma \rightharpoonup \Sigma)$

Where the state Σ is a partial function from identifiers to values: $\Sigma : \text{Ide} \to \mathbb{Z}$.

2.3 Axiomatic Semantics

Based on Floyd-Hoare Logic. It uses **Hoare Triples**:

$$\{P\}\ c\ \{Q\}$$

If the precondition P is true before c, then the postcondition Q will be true after the execution of c (assuming termination).

---

3. Meta-Language Properties

3.1 Termination

A program terminates if a reachable final result exists.

$$\forall a \in \text{Aexp}, \forall \sigma \in \Sigma, \exists n \in \mathbb{Z}.\ \langle a, \sigma \rangle \to n$$

3.2 Determinism

If the computation terminates, the result is unique.

$$\forall a, \sigma, n_1, n_2.\ (\langle a, \sigma \rangle \to n_1 \wedge \langle a, \sigma \rangle \to n_2) \implies n_1 = n_2$$

3.3 Equivalence

Two programs are equivalent if they are observationally indistinguishable.

- **Operational Equivalence ($\sim$):**

$$c_0 \sim c_1 \iff \forall \sigma, \sigma'. \, (\langle c_0, \sigma \rangle \to \sigma' \iff \langle c_1, \sigma \rangle \to \sigma')$$

- **Denotational Equivalence ($\equiv_{den}$):**

$$c_0 \equiv_{den} c_1 \iff \mathcal{C}[\![c_0]\!] = \mathcal{C}[\![c_1]\!]$$

3.4 Consistency

Fundamental theorem linking OS and DS. **Theorem 6.3:** For every command c and states σ, σ':

$$\langle c, \sigma \rangle \to \sigma' \iff \mathcal{C}[\![c]\!]\sigma = \sigma'$$

The proof requires two directions:

1. **Correctness:** $\langle c, \sigma \rangle \to \sigma' \implies \mathcal{C}[\![c]\!]\sigma = \sigma'$
2. **Completeness:** $\mathcal{C}[\![c]\!]\sigma = \sigma' \implies \langle c, \sigma \rangle \to \sigma'$

---

4. Logical Tools and Induction

4.1 Logical Systems

Defined by a set of **Inference Rules**:

$$\frac{\text{Premise}_1 \quad \ldots \quad \text{Premise}_n}{\text{Conclusion}} \quad \text{(Rule Name)}$$

A **Theorem** ($\vdash_R y$) is a formula for which a finite derivation (proof tree) based on the rules exists.

4.2 Unification

The process of finding a substitution σ (MGU - Most General Unifier) such that $\sigma t_1 = \sigma t_2$. Essential for applying SOS rules and in logic programming.

4.3 Induction Principles

All principles derive from well-founded induction.

Definition (Well-Founded Relation): A relation $\prec \subseteq A \times A$ is well-founded if there are no infinite descending chains ($a_0 \succ a_1 \succ \ldots$).

Theorem (Well-Founded Induction): To prove $\forall x \in A. \, P(x)$, it is sufficient to show:

$$forall x \in A. \, (\forall y \prec x. \, P(y)) \implies P(x)$$

Variants of Induction

Type	Scope	Description
Mathematical	$\mathbb{N}$	Base $P(0)$, Step $P(n) \implies P(n+1)$.

Type	Scope	Description
Structural	Syntactic Terms	Prove P for every constructor f, assuming P for sub-terms.
Rule Induction	Derivations/Theorems	Fundamental for inductive definitions (e.g., $\rightarrow$). If the property holds for the premises of every rule, it holds for the conclusion.

> **Deep Dive:** Structural induction fails on complex recursive constructs (or when syntax is circular). In those cases, rule induction on the derivation of the semantics is strictly necessary.

---

5. Functional Preliminaries (HOFL)

5.1 λ-Notation

Used as a meta-language for functions.

- **Abstraction:** $\lambda x.e$
- **α-conversion:** Renaming of bound variables ($\lambda x.x \equiv \lambda y.y$).
- **Substitution:** $t'[t/x]$ (must be *capture-avoiding*).

5.2 Haskell (Example)

Purely functional language used to exemplify referential transparency.

```haskell
-- Example: Palindrome check
-- Demonstrates declarative style and usage of higher-order functions
pal :: Eq a => [a] -> Bool
pal xs = (xs == reverse xs)
```

Math & Logic

This chapter formalizes the necessary tools to handle recursion, prove properties of infinite systems, and constructively define semantics.

---

1. Induction Principles

The core concept is **Well-Founded Induction**, from which all other principles are derived.

1.1 Well-Founded Relations

> **Definition (Well-Founded Relation):** A binary relation $\prec \subseteq A \times A$ is **well-founded** if it does not admit infinite descending chains $a_0 \succ a_1 \succ a_2 \ldots$.
>
> Equivalent to: **Minimal Element Principle**. $\prec$ is well-founded iff every non-empty subset $Q \subseteq A$ has a minimal element m ($\forall x \in Q.x \not\prec m$).

1.2 Principle of Well-Founded Induction (WFI)

Let $\prec$ be a well-founded relation on A. To prove $\forall a \in A.P(a)$:

$$\forall a \in A.\ (\forall b \prec a.P(b)) \implies P(a)$$

If we prove that the property holds for a assuming it is true for all its predecessors (Inductive Hypothesis), then it holds for the entire domain.

1.3 Rule Induction

When working with logical systems defined by inference rules (such as operational semantics), structural induction is insufficient (e.g., for `while`). We use induction on derivations.

Principle: Let I_R be the set of theorems derivable in a system R. To prove that a property P holds for every theorem $y \in I_R$:

$$\frac{\forall(\frac{x_1 \ldots x_n}{y}) \in R.\ (P(x_1) \wedge \cdots \wedge P(x_n)) \implies P(y)}{\forall \delta \in I_R.P(\delta)}$$

If the property is preserved by every rule (from premises to conclusion), then it holds for all derived theorems.

Practical Example: Determinacy of IMP We want to prove that if a command terminates, the result is unique. **Theorem:** $\langle c, \sigma \rangle \to \sigma_1 \wedge \langle c, \sigma \rangle \to \sigma_2 \implies \sigma_1 = \sigma_2$.

We define the property P on the derivation $\langle c, \sigma \rangle \to \sigma_1$ as:

$$P(\langle c, \sigma \rangle \to \sigma_1) \iff \forall \sigma_2.\ \langle c, \sigma \rangle \to \sigma_2 \implies \sigma_1 = \sigma_2$$

We apply rule induction:

1. **Skip Rule:** $\langle \textbf{skip}, \sigma \rangle \to \sigma$. If we have another derivation $\langle \textbf{skip}, \sigma \rangle \to \sigma_2$, the same rule must have been used (it is the only one for skip). Thus $\sigma_2 = \sigma$. (OK)
2. **Seq Rule:** $\frac{\langle c_0, \sigma \rangle \to \sigma'' \quad \langle c_1, \sigma'' \rangle \to \sigma'}{\langle c_0; c_1, \sigma \rangle \to \sigma'}$. *Inductive Hypothesis (IH):* Determinacy holds for the premises (execution of c_0 and c_1). *Step:* If $\langle c_0; c_1, \sigma \rangle \to \sigma_2$, the only applicable rule is Seq. Thus there exists σ_2'' such that $\langle c_0, \sigma \rangle \to \sigma_2''$ and $\langle c_1, \sigma_2'' \rangle \to \sigma_2$. By IH on c_0: $\sigma'' = \sigma_2''$. By IH on c_1: since the intermediate state is the same, $\sigma' = \sigma_2$. (OK)

---

2. Immediate Consequence Operator (ICO)

The set of theorems I_R of a logical system can be viewed as the fixpoint of a function. This connects logic to domain theory.

2.1 Definition of $\hat{R}$

Let F be the set of all possible formulas (facts). The operator $\hat{R} : \wp(F) \to \wp(F)$ computes the facts derivable in **one single step** from a set of hypotheses S.

$$\hat{R}(S) \triangleq \left\{ y \mid \exists \frac{x_1 \ldots x_n}{y} \in R \text{ such that } \{x_1, \ldots, x_n\} \subseteq S \right\}$$

2.2 Characterization Theorem

The operator $\hat{R}$ is **monotone** and **continuous** (if rules have finite premises). The set of theorems I_R coincides with the least fixpoint of $\hat{R}$:

$$I_R = \text{fix}(\hat{R}) = \bigcup_{n \in \mathbb{N}} \hat{R}^n(\emptyset)$$

- $\hat{R}^0(\emptyset) = \emptyset$
- $\hat{R}^1(\emptyset) =$ Axioms (rules without premises).
- $\hat{R}^{n+1}(\emptyset) =$ Facts derivable with proof trees of height $\leq n+1$.

2.3 Example: Balanced Parentheses

Consider the grammar for strings of balanced parentheses: $\epsilon \mid (s) \mid s_1 s_2$. Rules R:

1. $\overline{\epsilon}$ (Axiom)
2. $\frac{s}{(s)}$
3. $\frac{s_1 \quad s_2}{s_1 s_2}$

We compute the fixpoint (set of theorems) by iterating $\hat{R}$:

1. **Step 0:** $S_0 = \emptyset$.
2. **Step 1:** $\hat{R}(S_0) = \{\epsilon\}$ (Applying rule 1).
3. **Step 2:** $\hat{R}(S_1)$ uses $S_1 = \{\epsilon\}$.
   - Rule 2 on $\epsilon \to (\epsilon) = ()$.
   - Rule 3 on $\epsilon, \epsilon \to \epsilon\epsilon = \epsilon$.
   - $S_2 = \{\epsilon, ()\}$.
4. **Step 3:** $\hat{R}(S_2)$ uses $\{\epsilon, ()\}$.
   - Rule 2 on $() \to (())$.
   - Rule 3 on $s_1 = (), s_2 = () \to ()()$.
   - $S_3 = \{\epsilon, (), (()), ()()\}$.
5. **Limit:** The union $\bigcup_n S_n$ is the set of all balanced strings.

---

3. Partial Orders and CPO

3.1 Basic Definitions

- **Partial Order (PO):** Set P with a relation $\sqsubseteq$ that is reflexive, antisymmetric, and transitive.
- **Chain:** A sequence $\{d_n\}_{n\in\mathbb{N}}$ that is totally ordered: $d_0 \sqsubseteq d_1 \sqsubseteq d_2 \ldots$
- **Limit (LUB):** The *Least Upper Bound* of a chain, denoted as $\bigsqcup_n d_n$.

3.2 Complete Partial Orders (CPO)

Definition (CPO): A PO is a **CPO** if every chain has a limit (LUB) in D.

Definition (CPO$\perp$): A CPO is **pointed** (CPO$\perp$) if it possesses a minimal element, called **bottom** ($\perp$).

3.3 Functions on CPO

Let D, E be two CPOs. A function $f : D \to E$ can be:

1. **Monotone:** Preserves the order ($d \sqsubseteq d' \implies f(d) \sqsubseteq f(d')$).
2. **Continuous:** Preserves the limits of chains.

$$f\left(\bigsqcup_n d_n\right) = \bigsqcup_n f(d_n)$$

Note: Continuity implies monotonicity.

---

4. Kleene's Fixpoint Theorem

4.1 Statement

Let D be a CPO$\perp$ and $f : D \to D$ a **continuous** function. Then f has a **least fixed point**, computable as:

$$\text{fix}(f) = \bigsqcup_{n \in \mathbb{N}} f^n(\perp)$$

Where the approximation chain is: $\perp \sqsubseteq f(\perp) \sqsubseteq f(f(\perp)) \ldots$

4.2 Proof (Sketch)

1. **Chain:** Prove by mathematical induction that $f^n(\perp) \sqsubseteq f^{n+1}(\perp)$. (Base: $\perp \sqsubseteq f(\perp)$ is obvious).
2. **Fixpoint:** Let $d = \bigsqcup f^n(\perp)$. Applying f:

$$f(d) = f(\bigsqcup f^n(\perp)) = \bigsqcup f(f^n(\perp)) = \bigsqcup f^{n+1}(\perp) = d$$

   Continuity allows "moving the limit inside".
3. **Least:** If e is another fixpoint ($f(e) = e$), prove by induction that $f^n(\perp) \sqsubseteq e$, thus the limit $d \sqsubseteq e$.

---

5. Solved Exercises (Domain Theory)

Exercise: Continuity on Power Sets

Problem: Consider the CPO $(\wp(\mathbb{N}), \subseteq)$. Let $S \subseteq \mathbb{N}$ be a fixed set. Prove that the function $f_S : \wp(\mathbb{N}) \to \wp(\mathbb{N})$ defined as $f_S(X) = X \cap S$ is **continuous**.

Solution: To prove continuity, we must show that f_S preserves the limits of chains (Least Upper Bounds). In $(\wp(\mathbb{N}), \subseteq)$, the limit of a chain is the union of the sets: $\bigsqcup_i X_i = \bigcup_{i \in \mathbb{N}} X_i$.

We need to demonstrate that for every chain $\{X_i\}_{i \in \mathbb{N}}$:

$$f_S\left(\bigcup_{i \in \mathbb{N}} X_i\right) = \bigcup_{i \in \mathbb{N}} f_S(X_i)$$

Proof:

1. **LHS (Left Hand Side):**

$$f_S\left(\bigcup_i X_i\right) = \left(\bigcup_i X_i\right) \cap S$$

2. **RHS (Right Hand Side):**

$$\bigcup_i f_S(X_i) = \bigcup_i (X_i \cap S)$$

3. **Conclusion:** By the distributive property of intersection over union (including infinite union), we know that:

$$(\bigcup_i X_i) \cap S = \bigcup_i (X_i \cap S)$$

   Thus LHS = RHS. The function is continuous.

   Note: The same logic applies to $g_S(X) = X \cup S$ (using associativity/idempotence of union).

---

6. Calculus of Relations

The Calculus of Relations (CR) is an algebraic framework for reasoning about binary relations. It generalizes set theory and functions, providing a powerful language for abstract semantics.

6.1 Syntax and Operations

A relation R is a subset of the Cartesian product of two sets $A \times B$. The calculus defines the following operations on relations R, S:

- **Boolean Operations:**
  - **Union ($R \cup S$):** $x(R \cup S)y \iff xRy \vee xSy$.
  - **Intersection ($R \cap S$):** $x(R \cap S)y \iff xRy \wedge xSy$.
  - **Complement ($\neg R$ or $\bar{R}$):** $x\bar{R}y \iff \neg(xRy)$.
- **Relational Operations:**
  - **Composition ($R; S$):** $x(R; S)z \iff \exists y.xRy \wedge ySz$.
  - **Converse / Transpose (R° or R^{-1}):** $yR^\circ x \iff xRy$.
- **Constants:**
  - **Empty Relation ($\mathbb{O}$):** The relation containing no pairs (False).
  - **Universal Relation ($\Vdash$):** The relation containing all pairs (True).
  - **Identity ($\mathbb{I}$):** The diagonal relation $\{(x, x) \mid x \in A\}$.

6.2 Tarski's Axioms

Alfred Tarski axiomatized the calculus of relations. A **Relation Algebra** is a structure satisfying:

1. **Boolean Algebra:** The set of relations forms a Boolean algebra with $\cup, \cap, \neg, \mathbb{O}, \Vdash$.
2. **Associativity:** Composition is associative: $(R; S); T = R; (S; T)$.
3. **Identity:** $\mathbb{I}$ is the neutral element for composition: $R; \mathbb{I} = R = \mathbb{I}; R$.
4. **Involution:** Conversion is an involution: $(R^\circ)^\circ = R$ and $(R; S)^\circ = S^\circ; R^\circ$.
5. **Distributivity:** Composition distributes over union: $R; (S \cup T) = (R; S) \cup (R; T)$.
6. **Tarski's Rule:** $R \neq \mathbb{O} \implies \Vdash; R; \Vdash = \Vdash$ (If a relation is non-empty, it relates everything in the universal context).

6.3 Properties of Relations in CR

We can define standard properties purely algebraically:

Property	Definition in CR	Meaning
Reflexive	$\mathbb{I} \subseteq R$	$\forall x.xRx$
Transitive	$R; R \subseteq R$	$xRy \wedge yRz \implies xRz$
Symmetric	$R \subseteq R^\circ$ (or $R = R^\circ$)	$xRy \implies yRx$
Univalent (Functional)	$R^\circ; R \subseteq \mathbb{I}$	$xRy \wedge xRz \implies y = z$
Total	$\mathbb{I} \subseteq R; R^\circ$	$\forall x.\exists y.xRy$
Injective	$R; R^\circ \subseteq \mathbb{I}$	$xRy \wedge zRy \implies x = z$
Surjective	$\mathbb{I} \subseteq R^\circ; R$	$\forall y.\exists x.xRy$

> **Note:** A function is a relation that is both *Univalent* and *Total*. A bijection is a relation that is *Univalent, Total, Injective,* and *Surjective*.

6.4 Algebraic Structures and Laws

The set of relations CR_Σ equipped with its operations forms specific algebraic structures.

1. **Boolean Algebra:** $(CR_\Sigma, \cup, \cap, \neg, \mathbb{O}, \Vdash)$ is a Boolean Algebra.
2. **Monoid:** $(CR_\Sigma, ;, \mathbb{I})$ is a Monoid (Associativity of composition, Identity element).

3. **Semiring:** $(CR_\Sigma, \cup, \mathbb{O}, ;, \mathbb{I})$ forms an **Idempotent Semiring** (or Dioid).
   - $\cup$ is the addition (idempotent, commutative, with unit $\mathbb{O}$).
   - ; is the multiplication (associative, distributes over $\cup$, with unit $\mathbb{I}$ and zero $\mathbb{O}$).
   - **Distributivity Law:** $(e \cup f); g \equiv (e; g) \cup (f; g)$.
   - **Zero Law:** $e; \mathbb{O} \equiv \mathbb{O} \equiv \mathbb{O}; e$.

Involution Laws (Converse/Transpose): The conversion operator $(\cdot)^{op}$ is an involution that interacts with other operators: * **Involution:** $(e^{op})^{op} \equiv e$. * **Contravariance (Composition):** $(e; f)^{op} \equiv f^{op}; e^{op}$ (Order reversal). * **Homomorphism (Union):** $(e \cup f)^{op} \equiv e^{op} \cup f^{op}$. * **Interaction with Complement:** $\overline{(e^{op})} \equiv (\overline{e})^{op}$. * **Constants:** $\mathbb{I}^{op} \equiv \mathbb{I}$, $\mathbb{O}^{op} \equiv \mathbb{O}$, $\nVdash^{op} \equiv \nVdash$.

6.5 Semantics of Regular Expressions over Relations

We can define relations using regular expressions $e \in Reg_\Sigma$. The semantics of an expression can be seen as the union of the semantics of all individual "paths" or "traces" denoted by that expression.

Lemma (Trace Semantics): For all expressions $e \in Reg_\Sigma$ and for all interpretations $\mathcal{I}$:

$$[\![e]\!]_\mathcal{I} = \bigcup_{\nu \in [e]} [\![\nu]\!]_\mathcal{I}$$

Where $[e]$ is the set of strings (words) generated by the expression e.

Proof Sketch (by Structural Induction on e): * **Base Cases:** * $e = \emptyset$ (Empty set): $[\![\emptyset]\!]_\mathcal{I} = \emptyset = \bigcup_{v \in \emptyset} \dots$ * $e = \epsilon$ (Empty word): $[\![\epsilon]\!]_\mathcal{I} = \mathbb{I} = [\![\epsilon]\!]_\mathcal{I}$ (since $[\epsilon] = \{\epsilon\}$). * $e = a$ (Atom): $[\![a]\!]_\mathcal{I} = R_a = \bigcup_{v \in \{a\}} [\![v]\!]_\mathcal{I}$. * **Inductive Steps:** * **Union ($e = f + g$):** $[\![f + g]\!]_\mathcal{I} = [\![f]\!]_\mathcal{I} \cup [\![g]\!]_\mathcal{I}$. By IH: $(\bigcup_{\nu \in [f]} [\![\nu]\!]) \cup (\bigcup_{\mu \in [g]} [\![\mu]\!]) = \bigcup_{\xi \in [f] \cup [g]} [\![\xi]\!] = \bigcup_{\xi \in [f+g]} [\![\xi]\!]$. * **Concatenation ($e = f \cdot g$):** $[\![f \cdot g]\!]_\mathcal{I} = [\![f]\!]_\mathcal{I}; [\![g]\!]_\mathcal{I}$. Using distributivity of composition over union (infinite unions included in complete lattices): $(\bigcup_\nu [\![\nu]\!]); (\bigcup_\mu [\![\mu]\!]) = \bigcup_{\nu,\mu} ([\![\nu]\!]; [\![\mu]\!]) = \bigcup_{\xi \in [f \cdot g]} [\![\xi]\!]$.

6.6 Coreflexive Relations (Tests)

To model logical tests (guards) within the Calculus of Relations, we use **Coreflexive Relations**.

Definition: A relation R on set A is coreflexive if it is contained in the identity:

$$R \subseteq \mathbb{I}_A \iff \forall (x, y) \in R.\ x = y$$

We denote the set of coreflexives as $Cor(A)$.

Isomorphism with Predicates: There is a bijection between subsets of A (Predicates) and Coreflexives: 1. From Predicate to Relation: $c(P) = \{(x, x) \mid x \in P\}$. 2. From Relation to Predicate: $p(R) = \{x \mid (x, x) \in R\}$.

Properties of Tests: For coreflexive relations $R, S \in Cor(A)$, the boolean intersection coincides with sequential composition:

$$R \cap S = R; S$$

Proof Intuition: Since they are diagonal subsets, the only way to compose (x, x) and (y, y) is if $x = y$, which is exactly the intersection.

> **Role in KAT:** This property allows Kleene Algebra with Tests to treat logical conjunction $(b_1 \wedge b_2)$ as sequential composition $(b_1 \cdot b_2)$ within the algebraic framework.

IMP Semantics

This chapter defines the semantics of the **IMP** language, a minimal imperative language with static memory.

---

1. Syntax and States

Abstract Syntax

The language is defined by three syntactic categories (*sorts*):

- **Aexp:** Arithmetic Expressions (values in $\mathbb{Z}$)
- **Bexp:** Boolean Expressions (values in $\mathbb{B}$)
- **Com:** Commands (state transformations)

$$\begin{array}{lcl} a \in \text{Aexp} & ::= & n \mid x \mid a_0 + a_1 \mid a_0 - a_1 \mid \ldots \\ b \in \text{Bexp} & ::= & v \mid a_0 = a_1 \mid a_0 \leq a_1 \mid \neg b \mid b_0 \wedge b_1 \ldots \\ c \in \text{Com} & ::= & \textbf{skip} \mid x := a \mid c_0; c_1 \mid \textbf{if } b \textbf{ then } c_0 \textbf{ else } c_1 \mid \textbf{while } b \textbf{ do } c \end{array}$$

The State (Σ)

The state (or memory) is a function mapping identifiers to integer values.

$$\Sigma \stackrel{\text{def}}{=} \text{Loc} \to \mathbb{Z}$$

- **Update Notation:** $\sigma[n/x]$ denotes a state identical to σ except for the variable x, which now holds the value n.

---

2. Operational Semantics (Big-Step)

Defined via inference rules relating a program and an initial state to a final result (hiding intermediate steps). This style is also known as **Natural Semantics**.

Judgments

1. $\langle a, \sigma \rangle \to n$ (Aexp always terminates)
2. $\langle b, \sigma \rangle \to v$ (Bexp always terminates)
3. $\langle c, \sigma \rangle \to \sigma'$ (Convergence: the command terminates producing σ')

Inference Rules for Commands

Skip and Assignment:

$$\frac{}{\langle \textbf{skip}, \sigma \rangle \to \sigma}(\text{skip}) \qquad \frac{\langle a, \sigma \rangle \to m}{\langle x := a, \sigma \rangle \to \sigma[m/x]}(\text{assign})$$

Sequence:

$$\frac{\langle c_0, \sigma \rangle \to \sigma'' \quad \langle c_1, \sigma'' \rangle \to \sigma'}{\langle c_0; c_1, \sigma \rangle \to \sigma'}(\text{seq})$$

Conditional (If-Then-Else):

$$\frac{\langle b, \sigma \rangle \to \textbf{true} \quad \langle c_0, \sigma \rangle \to \sigma'}{\langle \textbf{if } b \ldots, \sigma \rangle \to \sigma'}(\text{iftt}) \quad \frac{\langle b, \sigma \rangle \to \textbf{false} \quad \langle c_1, \sigma \rangle \to \sigma'}{\langle \textbf{if } b \ldots, \sigma \rangle \to \sigma'}(\text{ifff})$$

While (Iteration):

$$\frac{\langle b, \sigma \rangle \to \textbf{false}}{\langle \textbf{while } b \textbf{ do } c, \sigma \rangle \to \sigma}(\text{whff}) \frac{\langle b, \sigma \rangle \to \textbf{true} \quad \langle c, \sigma \rangle \to \sigma'' \quad \langle \textbf{while } b \textbf{ do } c, \sigma'' \rangle \to \sigma'}{\langle \textbf{while } b \textbf{ do } c, \sigma \rangle \to \sigma'}(\text{whtt})$$

Note: The `whtt` rule is the only intrinsically recursive rule (the premise contains the same construct as the conclusion). This makes structural induction insufficient for proving properties about `while`; **rule induction** is required.

---

3. Denotational Semantics

Assigns meanings as mathematical functions. Since commands may diverge (infinite loop), we use domains with an undefined element (Bottom).

Domains and Functions

- **Lifted State ($\Sigma_\perp$):** $\Sigma \cup \{\perp\}$. The element $\perp$ represents non-termination.
- $\mathcal{A}[\![a]\!] : \Sigma \to \mathbb{Z}$
- $\mathcal{B}[\![b]\!] : \Sigma \to \mathbb{B}$
- $\mathcal{C}[\![c]\!] : \Sigma \to \Sigma_\perp$ (Total function returning $\perp$ if it diverges).

Lifting (Strict Extension)

To compose functions that may return $\perp$, we define the lifting operator $(\cdot)^*$: Given $f : \Sigma \to \Sigma_\perp$, its extension $f^* : \Sigma_\perp \to \Sigma_\perp$ is:

$$f^*(x) = \begin{cases} \perp & \text{if } x = \perp \\ f(x) & \text{if } x \in \Sigma \end{cases}$$

This is crucial for sequence: $\mathcal{C}[\![c_0; c_1]\!]\sigma = \mathcal{C}[\![c_1]\!]^*(\mathcal{C}[\![c_0]\!]\sigma)$.

Denotation of While (Fixpoint)

The `while` construct is defined as the **Least Fixed Point** (LFP) of a functional Γ.

$$\mathcal{C}[\![\textbf{while } b \textbf{ do } c]\!] = \text{fix}(\Gamma_{b,c})$$

Where $\Gamma_{b,c} : (\Sigma \to \Sigma_\perp) \to (\Sigma \to \Sigma_\perp)$ is defined as:

$$\Gamma_{b,c}(\varphi) = \lambda\sigma.\text{cond}(\mathcal{B}[\![b]\!]\sigma, \varphi^*(\mathcal{C}[\![c]\!]\sigma), \sigma)$$

Applying Kleene's Theorem:

$$\text{fix}(\Gamma) = \bigsqcup_{n \in \mathbb{N}} \Gamma^n(\perp)$$

Where $\Gamma^n(\perp)$ represents the semantics of the loop limited to n iterations.

---

4. Equivalence and Consistency

Operational Equivalence

Two commands are equivalent ($c_1 \sim c_2$) if they are observationally identical:

$$c_1 \sim c_2 \iff \forall \sigma, \sigma'.(\langle c_1, \sigma \rangle \to \sigma' \iff \langle c_2, \sigma \rangle \to \sigma')$$

Consistency Theorem (Theorem 6.3)

Operational Semantics (Big-Step) and Denotational Semantics coincide for every terminating program.

Statement:

$$\forall c \in \text{Com}, \forall \sigma, \sigma' \in \Sigma. \quad (\langle c, \sigma \rangle \to \sigma' \iff \mathcal{C}[\![c]\!]\sigma = \sigma')$$

The proof is divided into two parts (Correctness and Completeness).

Part 1: Correctness ($\Rightarrow$) Goal: Prove that $\langle c, \sigma\rangle \to \sigma' \implies \mathcal{C}[\![c]\!]\sigma = \sigma'$. **Technique:** Induction on the inference rules of the operational semantics (Rule Induction).

We define the property $P(\text{rule}) \iff \mathcal{C}[\![c]\!]\sigma = \sigma'$.

1. **Base Case (Skip):** Rule is $\langle \textbf{skip}, \sigma\rangle \to \sigma$. By denotational definition: $\mathcal{C}[\![\textbf{skip}]\!]\sigma = \sigma$. Thesis verified.
2. **Base Case (Assign):** Rule concludes $\langle x := a, \sigma\rangle \to \sigma[m/x]$ with premise $\langle a, \sigma\rangle \to m$. By consistency of expressions, $\mathcal{A}[\![a]\!]\sigma = m$. The denotational semantics defines $\mathcal{C}[\![x := a]\!]\sigma = \sigma[\mathcal{A}[\![a]\!]\sigma/x]$, which coincides with $\sigma[m/x]$.
3. **Inductive Step (Seq):** Rule: $\frac{\langle c_0,\sigma\rangle\to\sigma'' \quad \langle c_1,\sigma''\rangle\to\sigma'}{\langle c_0;c_1,\sigma\rangle\to\sigma'}$. Inductive Hypotheses: $\mathcal{C}[\![c_0]\!]\sigma = \sigma''$ and $\mathcal{C}[\![c_1]\!]\sigma'' = \sigma'$. Thesis:
$$\begin{aligned}\mathcal{C}[\![c_0; c_1]\!]\sigma &= \mathcal{C}[\![c_1]\!]^*(\mathcal{C}[\![c_0]\!]\sigma)\\ &= \mathcal{C}[\![c_1]\!]^*(\sigma'') \quad \text{(by IH on } c_0)\\ &= \mathcal{C}[\![c_1]\!]\sigma'' \quad \text{(since } \sigma'' \neq \bot)\\ &= \sigma' \quad \text{(by IH on } c_1)\end{aligned}$$
.
4. **Inductive Step (While - True Case):** Rule `whtt`. Assume $\mathcal{B}[\![b]\!]\sigma = \textbf{true}$. Inductive Hypotheses: $\mathcal{C}[\![c]\!]\sigma = \sigma''$ and $\mathcal{C}[\![\textbf{while } \ldots]\!]\sigma'' = \sigma'$. Thesis: Using the fixpoint property ($\text{fix}(\Gamma) = \Gamma(\text{fix}(\Gamma))$):
$$\begin{aligned}\mathcal{C}[\![w]\!]\sigma &= \mathcal{B}[\![b]\!]\sigma \to \mathcal{C}[\![w]\!]^*(\mathcal{C}[\![c]\!]\sigma), \sigma\\ &= \textbf{true} \to \mathcal{C}[\![w]\!]^*(\sigma''), \sigma\\ &= \mathcal{C}[\![w]\!]\sigma'' = \sigma' \quad \text{(by IH)}\end{aligned}$$

Part 2: Completeness ($\Leftarrow$) Goal: Prove that $\mathcal{C}[\![c]\!]\sigma = \sigma' \implies \langle c, \sigma\rangle \to \sigma'$ (assuming $\sigma' \neq \bot$). **Technique:** Structural Induction on c.

1. **Simple Cases:** For `skip`, `assign`, `seq`, and `if`, the proof is specular to correctness.
2. **While Case (`w`):** Structural induction fails because $\mathcal{C}[\![w]\!]$ is defined as a fixpoint. We use **Mathematical Induction** on the index n of the fixpoint approximations:
$$\mathcal{C}[\![w]\!]\sigma = \bigsqcup_n \Gamma^n(\bot)\sigma$$
We define property $A(n)$: $\forall \sigma, \sigma'. \Gamma^n(\bot)\sigma = \sigma' \implies \langle w, \sigma\rangle \to \sigma'$.
   - **Base ($n = 0$):** $\Gamma^0(\bot)\sigma = \bot$. Since the premise requires $\sigma' \neq \bot$, the implication is vacuously true.
   - **Step ($n + 1$):** Assume $A(n)$ holds. Let $\Gamma^{n+1}(\bot)\sigma = \sigma'$. Expanding Γ:
$$\Gamma^{n+1}(\bot)\sigma = \text{cond}(\mathcal{B}[\![b]\!]\sigma, \Gamma^n(\bot)^*(\mathcal{C}[\![c]\!]\sigma), \sigma)$$
     - If b is **false**: The result is σ. Rule `whff` confirms $\langle w, \sigma\rangle \to \sigma$.
     - If b is **true**: Then $\Gamma^n(\bot)(\sigma'') = \sigma'$, where $\sigma'' = \mathcal{C}[\![c]\!]\sigma$. By structural induction hypothesis on c: $\langle c, \sigma\rangle \to \sigma''$. By mathematical induction hypothesis on n: $\langle w, \sigma''\rangle \to \sigma'$. Applying rule `whtt`: $\langle w, \sigma\rangle \to \sigma'$.

---

5. Axiomatic Semantics (Hoare Logic)

Defines the meaning of commands via logical assertions. **Hoare Triple:** $\{P\}c\{Q\}$ *Meaning (Partial Correctness):* If P holds in the initial state and c terminates, then Q holds in the final state.

Inference Rules

1. **Skip:**

$$\frac{}{\{P\}\textbf{skip}\{P\}}$$

2. **Assignment (Backward):**

$$\frac{}{\{P[a/x]\}x := a\{P\}}$$

Example: To have $x > 10$ after $x := x + 1$, we must have $x + 1 > 10$ (i.e., $x > 9$) before.

3. **Sequence:**

$$\frac{\{P\}c_0\{R\} \quad \{R\}c_1\{Q\}}{\{P\}c_0; c_1\{Q\}}$$

4. **Conditional:**

$$\frac{\{P \wedge b\}c_0\{Q\} \quad \{P \wedge \neg b\}c_1\{Q\}}{\{P\}\textbf{if } b \textbf{ then } c_0 \textbf{ else } c_1\{Q\}}$$

5. **While:** P is called the loop **Invariant**.

$$\frac{\{P \wedge b\}c\{P\}}{\{P\}\textbf{while } b \textbf{ do } c\{P \wedge \neg b\}}$$

6. **Consequence (Weaken/Strengthen):**

$$\frac{P \implies P' \quad \{P'\}c\{Q'\} \quad Q' \implies Q}{\{P\}c\{Q\}}$$

---

6. Derivation Example (Swap)

Big-Step derivation for the variable swap program.

Initial State: $\sigma = \{x \mapsto 10, y \mapsto 20, z \mapsto 0\}$ **Program:** $c \equiv z := x; (x := y; y := z)$

Define intermediate states:

1. $\sigma_1 = \sigma[10/z] = \{x \mapsto 10, y \mapsto 20, z \mapsto 10\}$
2. $\sigma_2 = \sigma_1[20/x] = \{x \mapsto 20, y \mapsto 20, z \mapsto 10\}$
3. $\sigma_{final} = \sigma_2[10/y] = \{x \mapsto 20, y \mapsto 10, z \mapsto 10\}$

Derivation Tree:

$$\frac{\dfrac{\langle x, \sigma\rangle \to 10}{\langle z := x, \sigma\rangle \to \sigma_1}(\text{ass}) \quad \dfrac{\dfrac{\langle y, \sigma_1\rangle \to 20}{\langle x := y, \sigma_1\rangle \to \sigma_2}(\text{ass}) \quad \dfrac{\langle z, \sigma_2\rangle \to 10}{\langle y := z, \sigma_2\rangle \to \sigma_{final}}(\text{ass})}{\langle x := y; y := z, \sigma_1\rangle \to \sigma_{final}}(\text{seq})}{\langle z := x; (x := y; y := z), \sigma\rangle \to \sigma_{final}}(\text{seq})$$

—-

7. Solved Exercises (Exam)

Exercise: Efficiency Measure (Counting Guards)

Source: Exam 19/06/2019

Problem: We want to insert an efficiency measure into the operational semantics of IMP.

1. Redefine the semantics such that the judgment $\langle c, \sigma \rangle \to \sigma'$ becomes $\langle c, \sigma \rangle \xrightarrow{k} \sigma'$, where k is the exact number of boolean guards evaluated during execution.
2. Prove by induction that if the standard semantics converges, then there exists a k for which the new semantics converges.

Solution

1. New Operational Rules ($\xrightarrow{k}$) The idea is that every time we evaluate a `Bexp` (in `if` or `while`), the cost k increases by 1. Structural commands sum the costs of sub-commands.

- **Skip / Assign:** No guard evaluated.

$$\frac{}{\langle \textbf{skip}, \sigma \rangle \xrightarrow{0} \sigma} \qquad \frac{\langle a, \sigma \to n \rangle}{\langle x := a, \sigma \rangle \xrightarrow{0} \sigma[n/x]}$$

- **Sequence:** Sum of costs.

$$\frac{\langle c_0, \sigma \rangle \xrightarrow{k_1} \sigma'' \quad \langle c_1, \sigma'' \rangle \xrightarrow{k_2} \sigma'}{\langle c_0; c_1, \sigma \rangle \xrightarrow{k_1 + k_2} \sigma'}$$

- **If-Then-Else:** Cost 1 (for the guard) + cost of the chosen branch.

$$\frac{\langle b, \sigma \rangle \to \textbf{true} \quad \langle c_0, \sigma \rangle \xrightarrow{k} \sigma'}{\langle \textbf{if } b \ldots, \sigma \rangle \xrightarrow{k+1} \sigma'} \qquad \frac{\langle b, \sigma \rangle \to \textbf{false} \quad \langle c_1, \sigma \rangle \xrightarrow{k} \sigma'}{\langle \textbf{if } b \ldots, \sigma \rangle \xrightarrow{k+1} \sigma'}$$

- **While:**
  - *False Case:* Evaluate guard (1) and exit.

$$\frac{\langle b, \sigma \rangle \to \textbf{false}}{\langle \textbf{while } b \textbf{ do } c, \sigma \rangle \xrightarrow{1} \sigma}$$

  - *True Case:* Evaluate guard (1) + body cost (k_c) + subsequent iterations cost (k_w).

$$\frac{\langle b, \sigma \rangle \to \textbf{true} \quad \langle c, \sigma \rangle \xrightarrow{k_c} \sigma'' \quad \langle \textbf{while } \ldots, \sigma'' \rangle \xrightarrow{k_w} \sigma'}{\langle \textbf{while } b \textbf{ do } c, \sigma \rangle \xrightarrow{1 + k_c + k_w} \sigma'}$$

2. Proof (Rule Induction) We want to prove: $\langle c, \sigma \rangle \to \sigma' \implies \exists k \in \mathbb{N}.\ \langle c, \sigma \rangle \xrightarrow{k} \sigma'$. We define the property $P(\langle c, \sigma \rangle \to \sigma') \iff \exists k.\ \langle c, \sigma \rangle \xrightarrow{k} \sigma'$.

We apply **Rule Induction** on the standard semantics:

- **Seq Case:**
  - *Rule:* $\frac{\langle c_0, \sigma \rangle \to \sigma'' \quad \langle c_1, \sigma'' \rangle \to \sigma'}{\ldots}$
  - *Inductive Hypotheses:* There exist k_1, k_2 such that $\langle c_0, \sigma \rangle \xrightarrow{k_1} \sigma''$ and $\langle c_1, \sigma'' \rangle \xrightarrow{k_2} \sigma'$.
  - *Thesis:* We can apply the new semantics rule with cost $k = k_1 + k_2$. Thus a k exists. (Verified).
- **While-True Case:**
  - *Rule:* Premises $\langle b, \sigma \rangle \to \textbf{true}$, $\langle c, \sigma \rangle \to \sigma''$, $\langle w, \sigma'' \rangle \to \sigma'$.
  - *Inductive Hypotheses:* 1. $\exists k_c$ s.t. $\langle c, \sigma \rangle \xrightarrow{k_c} \sigma''$ 2. $\exists k_w$ s.t. $\langle w, \sigma'' \rangle \xrightarrow{k_w} \sigma'$
  - *Thesis:* Using the new while-true rule, we can derive the transition with cost $k = 1 + k_c + k_w$. Since $k_c, k_w \in \mathbb{N}$, then $k \in \mathbb{N}$. (Verified).

The other cases (`skip`, `assign`, `if`, `while-false`) are trivial or analogous.

---

8. Relational Semantics & Kleene Algebra with Tests (KAT)

We can abstract the semantics of IMP using **Kleene Algebra with Tests (KAT)**. This algebraic framework combines: * **Kleene Algebra ($K, +, \cdot, *, 0, 1$):** Models the control flow (nondeterminism, sequence, iteration). * **Boolean Algebra ($B, \vee, \wedge, \neg, 0, 1$):** Models the assertions/tests (guards).

8.1 Encoding IMP into KAT

We define a translation function $k : \text{Com} \to \text{KAT}$.

- **Skip:** $k(\textbf{skip}) = 1$ (Identity).
- **Abort:** $k(\textbf{abort}) = 0$ (Empty relation/Failure).
- **Assignment:** $k(x := e)$ is an atomic action.
- **Sequence:** $k(c_1; c_2) = k(c_1) \cdot k(c_2)$.
- **Conditional:**
$$k(\textbf{if } b \textbf{ then } c_1 \textbf{ else } c_2) = (b \cdot k(c_1)) + (\neg b \cdot k(c_2))$$
- **While Loop:**
$$k(\textbf{while } b \textbf{ do } c) = (b \cdot k(c))^* \cdot \neg b$$

8.2 Soundness & Completeness

The standard interpretation of KAT is the **Relational Model** ($\mathcal{P}(\Sigma \times \Sigma)$). **Theorem:** For any commands c, d:
$$k(c) =_{KAT} k(d) \implies c \sim d$$
(Algebraic equality implies semantic equivalence).

8.3 Schematic KAT (SKAT) Laws

Additional laws specific to assignments (useful for program transformation): 1. **Assignment Swap:**
$$(x := a \cdot y := b) = (y := b \cdot x := a)$$
Condition: $x \neq y$, $x \notin \text{vars}(b)$, $y \notin \text{vars}(a)$. 2. **Test Propagation:**
$$(x := a \cdot b) = (b[a/x] \cdot x := a)$$
Meaning: Checking b after assigning $x := a$ is equivalent to checking b with a substituted for x *before* the assignment. 3. **Variable Independence:**
$$b \cdot x := a = x := a \cdot b$$
Condition: $x \notin \text{vars}(b)$.

HOFL

HOFL (*Higher-Order Functional Language*) introduces functions as "first-class citizens". The complexity shifts from state management (which is absent) to the handling of complex types, recursion, and infinite domains.

---

1. Syntax and Types

1.1 Term Syntax

The grammar includes constructs for arithmetic, conditionals, pairs, functions, and recursion.
$$t ::= x \mid n \mid t_0 \text{ op } t_1 \mid \text{if } t \text{ then } t_0 \text{ else } t_1 \mid (t_0, t_1) \mid \text{fst}(t) \mid \text{snd}(t) \mid \lambda x.t \mid t_0 t_1 \mid \text{rec } x.t$$

1.2 Type System

Types are defined inductively:

$$\tau ::= \text{int} \mid \tau_0 \times \tau_1 \mid \tau_0 \to \tau_1$$

A term is **well-typed** $(t : \tau)$ if a derivation exists using the following inference rules $(\Gamma \vdash t : \tau)$:

- **Abstraction (abs):**

$$\frac{\Gamma, x : \tau_0 \vdash t : \tau_1}{\Gamma \vdash \lambda x.t : \tau_0 \to \tau_1}$$

- **Application (app):**

$$\frac{\Gamma \vdash t_1 : \tau_0 \to \tau_1 \quad \Gamma \vdash t_0 : \tau_0}{\Gamma \vdash t_1 t_0 : \tau_1}$$

- **Recursion (rec):**

$$\frac{\Gamma, x : \tau \vdash t : \tau}{\Gamma \vdash \text{rec } x.t : \tau}$$

---

2. Operational Semantics (Lazy)

In HOFL, there is no state (Environment Model). Evaluation is a relation $t \to c$ where c is a **Canonical Form**.

2.1 Canonical Forms (C_τ)

Values that require no further computation (final results).

1. **Integers:** $n \in \mathbb{Z}$.
2. **Pairs:** (t_0, t_1) where t_0, t_1 are closed terms (not necessarily evaluated in Lazy semantics).
3. **Abstractions:** $\lambda x.t$ (functions are values; the body is not evaluated until applied).

2.2 Inference Rules (Big-Step Lazy)

Arithmetic and Conditional:

$$\frac{t_0 \to n_0 \quad t_1 \to n_1}{t_0 \text{ op } t_1 \to n_0 \underline{\text{op}} n_1} \qquad \frac{t \to 0 \quad t_0 \to c_0}{\text{if } t \cdots \to c_0} \qquad \frac{t \to n \neq 0 \quad t_1 \to c_1}{\text{if } t \cdots \to c_1}$$

Pairs and Projections:

$$\frac{}{(t_0, t_1) \to (t_0, t_1)} \qquad \frac{t \to (t_0, t_1) \quad t_0 \to c_0}{\text{fst}(t) \to c_0}$$

Application (Lazy / Call-by-Name):

$$\frac{t_1 \to \lambda x.t_1' \quad t_1'[t_0/x] \to c}{(t_1 t_0) \to c}$$

Note: The argument t_0 is substituted into the function body **without being evaluated**.

---

3. Domain Theory

To define denotational semantics (especially for `rec`), domains must be **CPOs** (Complete Partial Orders) with a minimal element $\perp$ (bottom) representing non-termination.

3.1 Lifted Domains ($D_\perp$)

To distinguish between a computed result and non-termination, we use *lifted* domains.

$$D_\perp \triangleq \{\perp\} \cup \{\lfloor d \rfloor \mid d \in D\}$$

- $\perp$: No information (divergence).
- $\lfloor d \rfloor$: Defined value d (convergence).

3.2 Functions on Lifted Domains

1. **Lifting Function:** $\lfloor \cdot \rfloor : D \to D_\perp$. Maps $d \mapsto \lfloor d \rfloor$.
2. **Lifting Operator $(\cdot)^*$:** Extends a continuous function $f : D \to E$ to a function $f^* : D_\perp \to E$.

$$f^*(x) = \begin{cases} \perp_E & \text{if } x = \perp_{D_\perp} \\ f(d) & \text{if } x = \lfloor d \rfloor \end{cases}$$

3. **De-lifting (Let notation):** Syntactic sugar for handling lifted values.

$$(\text{let } x \leftarrow t.e) \equiv (\lambda x.e)^*(t)$$

If t diverges ($\perp$), the whole expression diverges. If $t = \lfloor d \rfloor$, it evaluates e with $x = d$.

---

4. Denotational Semantics

We define semantic domains D_τ for each type τ. In HOFL (Lazy), all domains are **Lifted** to allow divergence at every level.

4.1 Domain Definition

1. **Integers:** $D_{\text{int}} = \mathbb{Z}_\perp$ (Flat CPO).
2. **Pairs:** $D_{\tau_1 \times \tau_2} = (D_{\tau_1} \times D_{\tau_2})_\perp$.
   - *Why lifted?* To distinguish a diverging pair $\perp$ from a pair of diverging elements $(\perp, \perp)$.
3. **Functions:** $D_{\tau_1 \to \tau_2} = [D_{\tau_1} \to D_{\tau_2}]_\perp$.
   - Space of **continuous** functions between CPOs, lifted to distinguish the undefined function $\perp$ from the function that always diverges $\lambda x.\ \perp$.

4.2 Interpretation Function $[\![t]\!]\rho$

Maps terms and environments to elements of the domain: $[\![t]\!] : \text{Env} \to D_\tau$.

Basic Constructs:

- $[\![n]\!]\rho = \lfloor n \rfloor$
- $[\![x]\!]\rho = \rho(x)$
- $[\![t_1 \text{ op } t_2]\!]\rho = [\![t_1]\!]\rho \underline{\text{op}}_\perp [\![t_2]\!]\rho$ (Strict extension: if one is $\perp$, result is $\perp$).

Functional Constructs:

- **Abstraction (λ):**

$$[\![\lambda x.t]\!]\rho = \lfloor \lambda d.[\![t]\!]\rho[d/x] \rfloor$$

  The result is a defined element ($\lfloor \ldots \rfloor$) in the functional domain.
- **Application:**

$$[\![t_1 t_0]\!]\rho = \text{let } \varphi \leftarrow [\![t_1]\!]\rho.\varphi([\![t_0]\!]\rho)$$

  1. Evaluate t_1. If it diverges, return $\perp$.

2. If it converges to a function φ, apply φ to the denotation of the argument $[\![t_0]\!]\rho$ (without evaluating it first $\to$ Lazy).

Recursion (`rec`):

$$[\![\text{rec } x.t]\!]\rho = \text{fix}(\lambda d.[\![t]\!]\rho[d/x])$$

Computes the least fixed point of the function mapping x to the body t.

---

5. Consistency and Comparison

5.1 Correctness Theorem

Denotational semantics is consistent with operational semantics.

$$t \to c \implies \forall \rho.\ [\![t]\!]\rho = [\![c]\!]\rho$$

5.2 Lazy vs Eager (Unlifted)

In an **Eager** (Call-by-Value) language, the argument must be evaluated before application. This changes the domain structure.

Feature	Lazy (Standard HOFL)	Eager (CbyV)
App Rule	Substitutes un-evaluated t_0.	Evaluates $t_0 \to c_0$, then substitutes c_0.
Pair Domain	$(D_1 \times D_2)_\perp$ (Lifted)	$D_1 \times D_2$ (Unlifted - "smashed" product)
Function Domain	$[D_1 \to D_2]_\perp$	$[D_1 \to D_2]$
Divergence	`(\x. 1) (rec y. y)` $\to 1$	`(\x. 1) (rec y. y)` diverges.

> **Note on Unlifted Domains:** In Eager semantics, lifting is not needed on compound types because a term of type τ *always* denotes a defined value or diverges *before* returning. Domains become: $U_{\text{int}} = \mathbb{Z}_\perp$, $U_{\tau_1 \times \tau_2} = U_{\tau_1} \times U_{\tau_2}$.

Concurrency

This chapter moves from sequential models to concurrent ones. The focus is no longer on "computing a function", but on **interaction**, **non-determinism**, and **communication**.

---

1. Language Overview: Erlang vs Go

Both use *Message Passing*, but with opposite philosophies regarding memory and synchronization.

Feature	**Erlang**	**Go**
Concurrent Entity	Process (Actor) isolated (separate heap).	Goroutine (Lightweight Thread) with shared memory.
Communication	**Asynchronous** (Infinite Mailbox). Sender does not block.	**Synchronous** (default). `ch <- v` blocks until there is a receiver (Rendezvous).
Philosophy	"Let it crash" & Isolation.	"Do not communicate by sharing memory; share memory by communicating."
Reception	`receive` with Pattern Matching on the mailbox.	`<- ch` (select for multiple wait).

---

2. CCS (Calculus of Communicating Systems)

Process algebra introduced by Robin Milner. It is Turing-equivalent but focused on pure interaction (abstracts away from data values).

2.1 Process Syntax

$$P, Q ::= \mathbf{nil} \mid \alpha.P \mid P + Q \mid P \mid Q \mid P \setminus L \mid P[f] \mid A$$

- **Prefix ($\alpha.P$):** Performs action α and becomes P. α can be an input (a), an output ($\bar{a}$), or an internal action (τ).
- **Sum ($P + Q$):** Non-deterministic choice. If P acts, Q is discarded (and vice versa).
- **Parallel ($P \mid Q$):** P and Q execute concurrently (interleaving or synchronization).
- **Restriction ($P \setminus L$):** Actions in L (and their duals $\bar{L}$) become private: they cannot be performed externally but allow internal communication.
- **Identifier (A):** Used to define recursive processes (e.g., $A \stackrel{\text{def}}{=} \alpha.A$).

2.2 Operational Semantics (LTS)

Transitions are of the form $P \xrightarrow{\mu} P'$. The set of actions is $\text{Act} = \mathcal{L} \cup \{\tau\}$, where τ is the invisible (internal) action.

Dynamic Rules:

$$\frac{}{\alpha.P \xrightarrow{\alpha} P}(\text{Act}) \quad \frac{P \xrightarrow{\alpha} P'}{P + Q \xrightarrow{\alpha} P'}(\text{SumL}) \quad \frac{Q \xrightarrow{\alpha} Q'}{P + Q \xrightarrow{\alpha} Q'}(\text{SumR})$$

Static Rules (Parallel and Communication): Parallel composition allows interleaving (independent actions) or synchronization (handshake).

$$\frac{P \xrightarrow{\alpha} P'}{P|Q \xrightarrow{\alpha} P'|Q}(\text{ParL}) \quad \frac{Q \xrightarrow{\alpha} Q'}{P|Q \xrightarrow{\alpha} P|Q'}(\text{ParR})$$

Communication Rule (Synchronization): For communication to occur, P must perform an output (a) and Q an input ($\bar{a}$). The result is an internal action τ.

$$\frac{P \xrightarrow{a} P' \quad Q \xrightarrow{\bar{a}} Q'}{P|Q \xrightarrow{\tau} P'|Q'}(\text{Com})$$

Restriction Rule: Blocks actions in L, allowing only unrestricted actions or τ (result of internal communication) to pass.

$$\frac{P \xrightarrow{\mu} P' \quad \mu \notin L \cup \bar{L}}{P \setminus L \xrightarrow{\mu} P' \setminus L}(\text{Res})$$

2.3 Example of LTS Derivation

Consider the process $S \stackrel{\text{def}}{=} (a.P \mid \bar{a}.Q) \setminus \{a\}$. We want to derive the transition $S \xrightarrow{\tau} (P \mid Q) \setminus \{a\}$.

Derivation Tree:

$$\frac{\dfrac{\dfrac{}{\alpha.P \xrightarrow{a} P}(\text{Act}) \quad \dfrac{}{\bar{a}.Q \xrightarrow{\bar{a}} Q}(\text{Act})}{a.P|\bar{a}.Q \xrightarrow{\tau} P|Q}(\text{Com})}{(a.P|\bar{a}.Q)\setminus\{a\} \xrightarrow{\tau} (P|Q)\setminus\{a\}}(\text{Res})$$

Note: Without restriction, the process could also perform a (left only) or $\bar{a}$ (right only). Restriction "forces" synchronization by making individual communication attempts on a invisible.

---

3. Bisimulation

Trace equivalence (sequences of actions) is insufficient for concurrency because it ignores choice points (*branching structure*). Classic example: $a.(b+c)$ vs $a.b + a.c$ have the same traces $\{ab, ac\}$ but different behavior (moment of choice).

3.1 Formal Definition (Strong Bisimulation)

A binary relation $\mathcal{R} \subseteq \mathcal{P} \times \mathcal{P}$ is a **strong bisimulation** if, for every pair $(P, Q) \in \mathcal{R}$ and for every action α:

1. **Simulation P $\rightarrow$ Q:** If $P \xrightarrow{\alpha} P'$, then $\exists Q'$ such that $Q \xrightarrow{\alpha} Q'$ and $(P', Q') \in \mathcal{R}$.
2. **Simulation Q $\rightarrow$ P:** If $Q \xrightarrow{\alpha} Q'$, then $\exists P'$ such that $P \xrightarrow{\alpha} P'$ and $(P', Q') \in \mathcal{R}$.

Two processes are **Bisimilar** ($P \sim Q$) if there exists a bisimulation $\mathcal{R}$ containing the pair (P, Q).

3.2 The Game (Attacker vs Defender)

Bisimulation can be viewed as a turn-based game between **Alice (Attacker)** and **Bob (Defender)**.

- **State:** A pair of processes (P, Q).
- **Alice:** Wants to prove $P \not\sim Q$. Chooses one side (e.g., P) and makes a transition ($P \xrightarrow{\alpha} P'$).
- **Bob:** Must prove they are equivalent. Must respond on the other side (Q) with the *same* action ($Q \xrightarrow{\alpha} Q'$) trying to land in a state that remains equivalent.
- **Victory:** Alice wins if Bob cannot respond or if the new pair of states allows Alice to win in the future. Bob wins if the game continues infinitely (bisimulation).

3.3 Example: $a.(b+c)$ vs $a.b + a.c$

We prove that $P = a.(b.\mathbf{nil} + c.\mathbf{nil})$ and $Q = a.b.\mathbf{nil} + a.c.\mathbf{nil}$ are **NOT** bisimilar.

Alice's Winning Strategy:

1. **Alice** chooses Q (the non-deterministic process) and makes the left branch transition:

$$Q \xrightarrow{a} b.\mathbf{nil}$$

   We are now in the pair $(b.\mathbf{nil} + c.\mathbf{nil}, \quad b.\mathbf{nil})$.
2. **Bob** must respond with $P \xrightarrow{a}$. P has only one choice:

$$P \xrightarrow{a} b.\mathbf{nil} + c.\mathbf{nil}$$

   The new configuration is: $(b.\mathbf{nil} + c.\mathbf{nil}, \quad b.\mathbf{nil})$.
3. **Alice** now chooses the left process and performs action c:

$$(b.\mathbf{nil} + c.\mathbf{nil}) \xrightarrow{c} \mathbf{nil}$$

4. **Bob** must respond with the right process ($b.\mathbf{nil}$) performing c. But $b.\mathbf{nil}$ can only perform b. **Bob is stuck. Alice wins.** $\implies P \not\sim Q$.

3.4 Congruence

Strong bisimulation is a **congruence**: if $P \sim Q$, then $C[P] \sim C[Q]$ for any context C. This allows modular substitution of software components without altering system behavior.

---

4. Hennessy-Milner Logic (HML)

Modal logic for describing process properties. It is used as an alternative to bisimulation to distinguish processes.

4.1 Syntax

$$F ::= \text{tt} \mid \text{ff} \mid F_1 \wedge F_2 \mid F_1 \vee F_2 \mid \langle\alpha\rangle F \mid [\alpha]F$$

4.2 Semantics

- **Diamond ($\langle\alpha\rangle F$):** (Possibility / "Exists") It is *possible* to perform action α and end up in *at least one* state satisfying F.
$$P \models \langle\alpha\rangle F \iff \exists P'.(P \xrightarrow{\alpha} P' \wedge P' \models F)$$
- **Box ($[\alpha]F$):** (Necessity / "For all") *After every* action α, *all* resulting states must satisfy F.
$$P \models [\alpha]F \iff \forall P'.(P \xrightarrow{\alpha} P' \implies P' \models F)$$

  Note: If the process cannot perform α, $[\alpha]F$ is vacuously true.

  Duality: $[\alpha]F \equiv \neg\langle\alpha\rangle\neg F$.

4.3 Distinguishing Formulas (Example)

Let's revisit $P = a.(b + c)$ and $Q = a.b + a.c$. We want a formula F such that $P \models F$ and $Q \not\models F$.

The formula is: $[a](\langle b\rangle\mathbf{tt} \wedge \langle c\rangle\mathbf{tt})$ *Meaning:* "After every action a, I must be able to perform both b and c".

- **For P:** After the unique transition a, we arrive at $(b + c)$. This state can do b (true) and can do c (true). Thus P satisfies the formula.
- **For Q:**
  - If we take the first a, we arrive at $b.\mathbf{nil}$. This state satisfies $\langle b\rangle\text{tt}$ but **not** $\langle c\rangle\text{tt}$.
  - Since Box $[a]$ requires the property to hold for *all* states reachable by a, and one fails, Q does not satisfy the formula.

4.4 Hennessy-Milner Theorem

For finitely branching processes:

$$P \sim Q \iff (P \models F \iff Q \models F, \forall F \in \text{HML})$$

Two processes are bisimilar if and only if they satisfy the same logical HML formulas. If two processes are not bisimilar, there always exists an HML formula that distinguishes them.

---

6. CCS at Work: Modeling Patterns

This section explores how to use CCS primitives to model complex systems, data structures, and shared memory protocols.

6.1 Modular Construction of Buffers

We can build complex buffers by composing simple 1-position cells (B_0^1).

The Atomic Cell:

$$B_0^1 \stackrel{\text{def}}{=} \text{in}.B_1^1 \qquad B_1^1 \stackrel{\text{def}}{=} \overline{\text{out}}.B_0^1$$

Capacity 2 Buffer (Sequential/Chained): Constructed by linking two cells in series. The output of the first feeds the input of the second via a hidden internal channel (`mid`).

$$B_2 \stackrel{\text{def}}{=} (B_0^1[\text{mid}/\text{out}] \mid B_0^1[\text{mid}/\text{in}]) \setminus \{\text{mid}\}$$

Behavior: It can accept two `in` actions before blocking, but preserves FIFO order.

Capacity 2 Buffer (Parallel): Constructed by placing two cells in parallel without linking them.

$$B_{par} \stackrel{\text{def}}{=} B_0^1 \mid B_0^1$$

Behavior: It accepts two `in` actions, but does **not** guarantee order (race condition on `out`).

6.2 Encoding Shared Memory (Variables)

Since CCS has no mutable state, variables are modeled as **processes** that serve read/write requests.

The Variable Process: A variable X storing value v (from a domain V) is a process that: 1. Accepts a read request ($\overline{\text{xr}}$) and sends v. 2. Accepts a write request (xw_k) and becomes X storing k.

$$X_v \stackrel{\text{def}}{=} \overline{\text{xr}}_v.X_v + \sum_{k \in V} \text{xw}_k.X_k$$

Using the Variable: * **Read (x):** The client performs xr_v (input) to synchronize with the variable's output. * **Assign ($x := k$):** The client performs $\overline{\text{xw}}_k$ (output) to trigger the variable's state change. * **Restriction:** The channels $\{\text{xr}, \text{xw}\}$ must be restricted to enforce synchronization between the specific client and the variable process.

6.3 Case Study: Peterson's Mutual Exclusion

We can verify the correctness of Peterson's algorithm by modeling the agents and the memory.

Architecture: The system consists of two processes (P_1, P_2) and three shared variables (b_1, b_2, k) running in parallel.

$$Sys \stackrel{\text{def}}{=} (P_1 \mid P_2 \mid B_{1f} \mid B_{2f} \mid K_1) \setminus \mathcal{L}_{internal}$$

Process Logic (P_1): 1. **Enter Protocol:** Write `true` to b_1, write `2` to k. 2. **Busy Wait:** Loop while (b_2 is true AND k is 2). 3. **Critical Section:** Perform task (e.g., `enter1`, `exit1`). 4. **Exit Protocol:** Write `false` to b_1.

Formal Verification (Recursive HML): We can define properties using recursive equations (precursor to μ-calculus).

1. **Mutual Exclusion (Safety):** It must never be the case that both P_1 and P_2 are in the critical section.

$$Safe \stackrel{\text{def}}{=} [\text{enter}_1]([\text{enter}_2]\text{ff}) \wedge [-]Safe$$

   Meaning: If P_1 enters, then P_2 cannot enter immediately. This must hold forever ($[-]Safe$).

2. **Liveness (No Deadlock):** It is always possible to perform an action.

$$Live \stackrel{\text{def}}{=} \langle - \rangle \text{tt} \wedge [-]Live$$

3. **Bounded Overtaking:** If P_1 wants to enter, P_2 cannot enter infinitely many times before P_1.

Real Languages

This chapter bridges theoretical models (HOFL, CCS) with real-world programming languages, analyzing how abstract concepts translate into concrete implementations.

---

1. Haskell: Pure Functional Programming

Haskell is the practical counterpart of **HOFL**. It is a **purely functional** language (no side-effects, referential transparency) and **lazy** (lazy evaluation).

1.1 Syntax and Functions

Unlike HOFL, Haskell offers advanced syntactic constructs for defining functions.

- **Pattern Matching:**

```haskell
fac 0 = 1
fac n = n * fac (n-1)
```

- **Guards (`|`):** Alternative to nested `if-then-else`.

```haskell
max a b
  | a > b     = a
  | otherwise = b
```

- **Partial Application:** Every function is curried by default.

```haskell
add :: Int -> Int -> Int
add x y = x + y

inc = add 1 -- inc :: Int -> Int (Partial Application)
```

1.2 Data Types and Type Classes

Haskell extends HOFL's type system with user-defined types and overloading.

- Defining New Data Types (data):

  Similar to sum/product types, but named.

```haskell
-- A recursive parametric 'Tree' type 'a'
data Tree a = Leaf a | Node (Tree a) (Tree a)
```

- Type Classes (class, instance):

  Mechanism for overloading (ad-hoc polymorphism). Does not exist in HOFL.

```haskell
-- Class definition (interface)
class Eq a where
    (==) :: a -> a -> Bool
    (/=) :: a -> a -> Bool

-- Instance for the Bool type
instance Eq Bool where
    True == True = True
    False == False = True
    _ == _ = False
```

1.3 Lazy Evaluation and Lifted Domains

The defining feature of Haskell is that function arguments are not evaluated until they are strictly necessary.

Theory (HOFL):

This corresponds to the use of **Lifted Domains** ($D_\perp$) in denotational semantics.

- The application rule is $[\![t_1 t_0]\!]\rho = \text{let } \varphi \Leftarrow [\![t_1]\!]\rho.\ \varphi([\![t_0]\!]\rho)$.
- The argument $[\![t_0]\!]\rho$ is passed to the function *without being de-lifted* (without checking if it is $\perp$).

Practice (Haskell):

This allows defining **infinite data structures** (Streams).

```
-- Infinite list of natural numbers
nats = [0 .. ]

-- Fibonacci: defined recursively on the list itself
-- zip and tail work because the list does not need to be fully in memory
fib :: Num b => [b]
fib = 1 : 1 : [ x+y | (x,y) <- zip fib (tail fib) ]
```

> **Comparison:** In an *Eager* (Call-by-Value) language, the definition of `fib` above would cause an immediate infinite loop (stack overflow) by trying to evaluate the argument of `zip` before calling the function.

1.4 Solved Exercises

1. Prime Numbers (Infinite Sieve) Uses laziness to filter an infinite list.

```
factors n = filter (\d -> n `mod` d == 0) [1..n]
prime n = factors n == [1, n]
primes = filter prime [2..] -- Infinite list of primes
```

---

2. Erlang: Actor Model and Distributed Concurrency

Erlang implements the **Actor** model, where each process is an isolated entity that shares no memory and communicates only via asynchronous messages.

2.1 Syntax and Primitives

- **Spawn:** `Pid = spawn(fun)`. Creates a new lightweight process and returns its PID.
- **Send (!):** `Pid ! Msg`. Sends a message **asynchronously**. The sender never blocks (non-blocking).
- Receive with Timeout:

  Extracts messages from the mailbox (infinite local FIFO queue). The after clause handles the timeout.

  ```
  receive
      {From, Msg} ->
          From ! {self(), ok};
      Stop ->
          true
  after 1000 ->
      io:format("Timeout!~n")
  end.
  ```

2.2 Example: Counter Server (CCS Recursion)

In Erlang, mutable state is managed via **tail recursion**, exactly like in CCS (`rec X. P`).

```
-module(counter).
-export([start/0, loop/1]).

start() -> spawn(counter, loop, [0]).

loop(Val) ->
    receive
        inc -> loop(Val + 1);  % Tail call (updated state)
        {From, val} ->
            From ! {self(), Val},
            loop(Val);
        stop -> true
    end.
```

CCS Mapping: $C(n) \stackrel{\text{def}}{=} \text{inc}.C(n+1) + \text{val}.\overline{v_n}.C(n)$.

---

3. Go: Goroutines and Channels (CSP)

Go is inspired by **CSP** (Communicating Sequential Processes).

3.1 Channels and Directionality

Channels are typed and can be unidirectional for safety.

- `chan T`: Bidirectional (read/write).
- `chan<- T`: **Send-only** (write only).
- `<-chan T`: **Receive-only** (read only).

```
// Pipeline example
func producer(out chan<- int) {
    out <- 1
    close(out)
}

func consumer(in <-chan int) {
    msg := <-in
    println(msg)
}
```

3.2 Select (Non-Deterministic Choice)

The `select` construct allows a goroutine to wait on multiple channels simultaneously. Corresponds to the sum + of CCS, but with input/output guards.

- If multiple cases are ready, one is chosen **pseudo-randomly**.
- If no case is ready, it blocks (or executes `default` if present).

```
select {
case msg1 := <-c1:
    println("Received", msg1)
case c2 <- 42:
```

```
    println("Sent 42")
default:
    println("No communication ready (non-blocking)")
}
```

3.3 Channels: Unbuffered vs Buffered

Type	Syntax	Corresponding CCS Semantics
Unbuffered	`make(chan T)`	**Synchronous** (Handshake). $\alpha.\mathbf{nil} \mid \bar{\alpha}.\mathbf{nil} \xrightarrow{\tau} \mathbf{nil}$. Sender and receiver must synchronize (Rendezvous).
Buffered	`make(chan T, N)`	**Asynchronous** (up to N). The sender does not block until the buffer is full.

3.4 Example: Deadlock in Go (vs CCS)

A deadlock in Go on unbuffered channels is identical to a CCS process blocked on an action without a partner.

```go
package main
func main() {
    ch := make(chan int) // Synchronous
    ch <- 42 // DEADLOCK! No one is listening.
    // In CCS: 'a.nil (without the partner in parallel)
}
```

Advanced Concurrency

This chapter extends the CCS theory to handle abstraction (ignoring internal τ actions) and introduces temporal logics for the formal verification of properties.

---

1. Weak Bisimulation

Strong bisimulation ($\sim$) is often too restrictive: it distinguishes processes that have the same observable behavior but differ in internal τ (tau) actions. Example: A buffer that performs an internal τ step to move data ($a.\tau.\bar{b}$) should be equivalent to an ideal buffer ($a.\bar{b}$), but $\sim$ distinguishes them.

1.1 Weak Transition ($\Longrightarrow$)

We define a new transition relation that "skips" τ actions.

- **Silent Weak Transition ($\overset{\tau}{\Longrightarrow}$):** A sequence of zero or more τ steps.

$$p \overset{\tau}{\Longrightarrow} q \iff p(\xrightarrow{\tau})^* q$$

  (Note: $p \overset{\tau}{\Longrightarrow} p$ is always true).

- **Observable Weak Transition ($\overset{\alpha}{\Longrightarrow}$):** An action α "surrounded" by τ steps.

$$p \overset{\alpha}{\Longrightarrow} q \iff \exists p', q'.\ p \overset{\tau}{\Longrightarrow} p' \xrightarrow{\alpha} q' \overset{\tau}{\Longrightarrow} q$$

  Meaning: do some internal work, then the visible action, then more internal work.

1.2 Definition of Weak Bisimulation ($\approx$)

A relation $\mathcal{R}$ is a weak bisimulation if, for every $(p, q) \in \mathcal{R}$:

1. If $p \xrightarrow{\alpha} p'$, then $\exists q'$ such that $q \overset{\alpha}{\Longrightarrow} q'$ and $(p', q') \in \mathcal{R}$.
2. If $p \xrightarrow{\tau} p'$, then $\exists q'$ such that $q \overset{\tau}{\Longrightarrow} q'$ and $(p', q') \in \mathcal{R}$. *(Note: here q can respond by staying still, since $\overset{\tau}{\Longrightarrow}$ includes zero steps).*

Two processes are **Weakly Bisimilar** ($p \approx q$) if there exists a weak bisimulation containing them.

1.3 The Congruence Problem (Summation)

Weak bisimulation is **NOT a congruence** with respect to the sum operator ($+$).

Counterexample 1 (Choice Resolution): Let $P = a.\mathbf{nil}$ and $Q = \tau.a.\mathbf{nil}$. Clearly $P \approx Q$ (the initial τ is weakly unobservable). However, in the context $C[\cdot] = \cdot + b.\mathbf{nil}$: * $C[P] = a.\mathbf{nil} + b.\mathbf{nil}$ (can do a or b). * $C[Q] = \tau.a.\mathbf{nil} + b.\mathbf{nil}$. $C[Q]$ can perform a silent step τ to become $a.\mathbf{nil}$, effectively **discarding** the option b. $C[P]$ cannot mimic this loss of capability silently. Thus $C[P] \not\approx C[Q]$.

Counterexample 2 (Silent Divergence): Weak bisimulation cannot distinguish between a deadlock and a silent loop (divergence). Let $D \overset{\text{def}}{=} \text{rec } x.\tau.x$ (Infinite τ-loop). $D \approx \mathbf{nil}$. But if we place them in a sum, the behavior might differ depending on fairness assumptions (though in standard CCS, this specific case is less critical for congruence than the choice resolution).

1.4 Observational Congruence (Weak Congruence)

To restore compositionality, we define **Observational Congruence** ($\cong$). Two processes p, q are congruent if: 1. They are weakly bisimilar ($p \approx q$). 2. **Root Condition:** For every α (including τ), if $p \xrightarrow{\alpha} p'$ then $q \overset{\alpha}{\Longrightarrow} q'$ (with at least one τ step if $\alpha = \tau$). *Ideally:* "Bob cannot stay idle on the very first move".

1.5 Milner's τ-Laws

These laws hold for observational congruence and are useful for algebraic simplification of processes.

1. $\alpha.\tau.P \cong \alpha.P$ (Absorption)
2. $P + \tau.P \cong \tau.P$ (Guard)
3. $\alpha.(P + \tau.Q) + \alpha.Q \cong \alpha.(P + \tau.Q)$ (Distributivity)

Exercise: Proof of $P + \tau.P \approx \tau.P$ We show that $\mathcal{R} = \{(P + \tau.P, \tau.P) \mid P \in \mathcal{P}\} \cup \text{Id}$ is a weak bisimulation. * **Left to Right:** * If $P + \tau.P \xrightarrow{\tau} P$ (right branch), $\tau.P \overset{\tau}{\Longrightarrow} P$ (via one τ). Pair $(P, P) \in \text{Id}$. * If $P + \tau.P \xrightarrow{\alpha} P'$ (left branch), $\tau.P \xrightarrow{\tau} P \xrightarrow{\alpha} P'$, so $\tau.P \overset{\alpha}{\Longrightarrow} P'$. Pair $(P', P') \in \text{Id}$. * **Right to Left:** * If $\tau.P \xrightarrow{\tau} P$, then $P + \tau.P \xrightarrow{\tau} P$. Pair $(P, P) \in \text{Id}$.

---

2. Temporal Logics (LTL & CTL)

Temporal logics allow specifying properties such as "Safety" (nothing bad ever happens) and "Liveness" (something good eventually happens).

2.1 LTL (Linear Temporal Logic)

Interprets time as a straight line (a single execution trace). Formulas are evaluated over a path $\pi = s_0 \to s_1 \to s_2 \ldots$

Operators:

- **X (Next):** $\pi \models \mathbf{X}\phi$ if ϕ holds in s_1.
- **G (Globally):** $\pi \models \mathbf{G}\phi$ if ϕ holds in all s_i (Safety).

- **F (Finally):** $\pi \models \mathbf{F}\phi$ if $\exists i$ s.t. ϕ holds in s_i (Liveness).
- **U (Until):** $\pi \models \phi\mathbf{U}\psi$ if ψ holds eventually, and ϕ holds until that moment.

2.2 CTL (Computation Tree Logic)

Interprets time as a tree (Branching Time). In each state, multiple futures are possible. Every temporal operator must be preceded by a path quantifier.

Quantifiers:

- **A (All):** For all future paths.
- **E (Exists):** There exists at least one future path.

CTL Examples:

- **AG**(safe): In all paths, in all states, I am safe (Strong Invariant).
- **EF**(start): There exists a path where eventually I start (Reachability).
- **AF**(end): In all paths, eventually I finish (Guaranteed Termination).

2.3 Expressiveness Comparison

- **LTL: FG**p (On this path, eventually p will be continuously true).
- **CTL: AGEF**p (It is always possible to reset the system and reach p).
- They are incomparable: there are LTL properties not expressible in CTL and vice versa.

---

3. The μ-Calculus

It is the most powerful logic (subsumes LTL and CTL). It is based on fixpoints.

3.1 Basic Syntax

$$\phi ::= \mathrm{tt} \mid \mathrm{ff} \mid Z \mid \phi_1 \wedge \phi_2 \mid \phi_1 \vee \phi_2 \mid \langle\alpha\rangle\phi \mid [\alpha]\phi \mid \mu Z.\phi \mid \nu Z.\phi$$

3.2 Fixpoints and Properties

- $\mu Z.\phi$ **(Least Fixed Point):** Used for **Liveness** / Reachability.
  - Example: $\mu Z.(p \vee \langle\cdot\rangle Z) \to$ "Either p holds now, or I can take a step and retry". (Corresponds to **EF**p).
  - *Intuition:* Looks for a "finite proof" (a path ending in p).
- $\nu Z.\phi$ **(Greatest Fixed Point):** Used for **Safety** / Invariants.
  - Example: $\nu Z.(p \wedge [\cdot]Z) \to$ " p holds now AND after every step Z still holds". (Corresponds to **AG**p).
  - *Intuition:* Allows infinite loops where p is always true.

---

4. Formal Semantics of Temporal Logics

4.1 LTL Semantics (Linear Time)

LTL formulas are evaluated over a linear structure $S : P \to \wp(\mathbb{N})$ representing when atomic propositions hold. Let S^k denote the time shifted by k steps.

Satisfaction Relation ($S \models \psi$): * $S \models \mathbf{X}\psi \iff S^1 \models \psi$ (Next) * $S \models \mathbf{F}\psi \iff \exists k.S^k \models \psi$ (Finally) * $S \models \mathbf{G}\psi \iff \forall k.S^k \models \psi$ (Globally) * $S \models \psi_1\mathbf{U}\psi_2 \iff \exists k.(S^k \models \psi_2 \wedge \forall i < k.S^i \models \psi_1)$ (Until)

Derived Equivalences: * $\mathbf{F}\psi \equiv \mathrm{tt}\mathbf{U}\psi$ * $\mathbf{G}\psi \equiv \neg\mathbf{F}\neg\psi$

4.2 CTL Semantics (Branching Time)

CTL is defined on a branching structure (infinite tree). It relies on a minimal set of operators: $EX, EG, E(\cdot U \cdot)$.

Derived Operators: * $\mathbf{AX}\psi \equiv \neg\mathbf{EX}\neg\psi$ (All Next) * $\mathbf{AF}\psi \equiv \neg\mathbf{EG}\neg\psi$ (All Finally) * $\mathbf{AG}\psi \equiv \neg\mathbf{EF}\neg\psi$ (All Globally)

5. Specification Examples

5.1 Shared Resource

System with 2 processes (p_1, p_2) accessing a resource r. Atoms: req_i (request), use_i (access), rel_i (release).

1. **Mutual Exclusion:** Only one uses r at a time. $\mathbf{G}\neg(use_1 \wedge use_2)$
2. **No Starvation:** If p_1 requests, it eventually accesses. $\mathbf{G}(req_1 \implies \mathbf{F}use_1)$
3. **Priority:** If both request, p_1 gets access first. $\mathbf{G}((req_1 \wedge req_2) \implies (\neg use_2 \mathbf{U}(use_1 \wedge \neg use_2)))$

5.2 The "Dogs" Problem

Three dogs, two couches, one garden. Predicates: $couch_{i,j}$ (dog i on couch j), $garden_i$ (dog i in garden).

1. **LTL:** If dog 1 is in garden, he stays there until he sits on a couch (or stays forever). $\mathbf{G}(garden_1 \implies ((\mathbf{G}garden_1) \vee (garden_1 \mathbf{U}(couch_{1,1} \vee couch_{1,2}))))$
2. **CTL:** If couch 1 is occupied (by dog 1 or 3), dog 2 eventually goes to garden. $\mathbf{AG}((couch_{1,1} \vee couch_{3,1}) \implies \mathbf{AF}garden_2)$

6. Mu-Calculus Deep Dive

The μ-calculus is the most expressive temporal logic, subsuming LTL, CTL, and CTL*. Its semantics is based on the complete lattice of sets of states $(\wp(V), \subseteq)$. Since all operators are monotonic, the Knaster-Tarski theorem guarantees the existence of fixpoints.

6.1 Syntax & Positive Normal Form

Formulas are typically required to be in **Positive Normal Form** (negations only appear directly before atomic propositions). This ensures monotonicity.

Negation Rules (De Morgan for Fixpoints): * $\neg\mu X.\phi(X) \equiv \nu X.\neg\phi(\neg X/X)$ * $\neg\nu X.\phi(X) \equiv \mu X.\neg\phi(\neg X/X)$ * $\neg\langle\alpha\rangle\phi \equiv [\alpha]\neg\phi$

6.2 Fixpoint Computation (Example)

Let us compute the denotation of a formula on a finite LTS. **Formula:** $\Phi = \mu X.((p \wedge [\cdot]X) \vee (\neg p \wedge \langle\cdot\rangle X))$. * *Intuitive Meaning:* "Eventually reach a 'safe' state where p holds and all successors are safe, OR if p doesn't hold, keep moving". This is a hybrid property combining Liveness (μ) and Safety ($[\cdot]$).

LTS Setup: * States: $V = \{s_1, s_2, s_3, s_4\}$. * Transitions: $s_1 \to s_2$, $s_1 \to s_3$, $s_2 \to s_3$, $s_3 \to s_3$ (loop), $s_3 \to s_4$, $s_4 \to \ldots$ (deadlock or external). * Predicate: $P = \{s_4\}$ (only s_4 satisfies p).

Iterative Calculation ($[\![\Phi]\!] = \bigcup_n f^n(\emptyset)$): Since it is a Least Fixpoint (μ), we start from the empty set $\emptyset$.

1. **Step 0:** $S_0 = \emptyset$.
2. **Step 1:** Evaluate body with $X = \emptyset$.
   - Term 1: $p \wedge [\cdot]\emptyset \implies \{s_4\} \cap \{v \mid \forall w.v \to w \implies w \in \emptyset\}$. s_4 has no outgoing transitions (or only to $\emptyset$), so $[\cdot]\emptyset$ is true for s_4. $\to \{s_4\}$.
   - Term 2: $\neg p \wedge \langle\cdot\rangle\emptyset \implies \{s_1, s_2, s_3\} \cap \emptyset = \emptyset$.
   - **Result:** $S_1 = \{s_4\}$.
3. **Step 2:** Evaluate body with $X = \{s_4\}$.
   - Term 1: $p \wedge [\cdot]\{s_4\} \to \{s_4\} \cap \{s \mid \text{successors}(s) \subseteq \{s_4\}\}$. ($s_4$ is trivial). $\to \{s_4\}$.

- Term 2: $\neg p \wedge \langle\cdot\rangle\{s_4\} \to \{s_1, s_2, s_3\} \cap \{s \mid \exists w \in \{s_4\}.s \to w\}$. Only s_3 has a transition to s_4. $\to \{s_3\}$.
- **Result:** $S_2 = \{s_3, s_4\}$.

4. **Step 3:** Evaluate body with $X = \{s_3, s_4\}$.
   - Term 2: $\langle\cdot\rangle\{s_3, s_4\}$. Now s_1 can reach s_3, and s_2 can reach s_3.
   - **Result:** $S_3 = \{s_1, s_2, s_3, s_4\}$.
5. **Step 4:** Stability reached ($S_4 = S_3$). The set of states satisfying Φ is the entire set V.

6.3 Alternation Depth

The power of μ-calculus lies in nesting fixpoints. * **Alternation Depth (AD):** The number of alternating nestings between μ and ν. * $AD = 0$: HML (no recursion). * $AD = 1$: CTL (simple recursion, e.g., $\mathbf{AG}p = \nu X.p \wedge [\cdot]X$). * $AD = 2$: LTL and CTL* (properties like fairness $\mathbf{GF}p$).

Exam Questions & Answers (Part 1)

This section covers the mathematical foundations: Logic, Unification, and Induction principles.

Part 1: Logic, Unification & Induction

Questions

1. **Signature:** What is a signature Σ? A1.1|Answer
2. **Terms:** What is a term over a signature? A1.2|Answer
3. **Substitution:** What is a substitution? A1.3|Answer
4. **Instantiation:** What is the instantiation order on substitutions? A1.4|Answer
5. **Unification Problem:** What is the Unification Problem? A1.5|Answer
6. **MGU:** What is a Most General Unifier (MGU)? A1.6|Answer
7. **Algorithm:** Describe the Unification Algorithm (Martelli-Montanari). A1.7|Answer
8. **Logical System:** What is a Logical System? A1.8|Answer
9. **Derivations:** What is a derivation rule and a derivation tree? A1.9|Answer
10. **Goal-oriented:** What is a goal-oriented derivation? A1.10|Answer
11. **Well-Foundedness:** What is a well-founded relation? A1.11|Answer
12. **WFI Principle:** What is the Principle of Well-Founded Induction? A1.12|Answer
13. **Structural Induction:** What is Structural Induction? A1.13|Answer
14. **Rule Induction:** What is Rule Induction? A1.14|Answer
15. **Application:** How to prove properties of the operational semantics using Rule Induction? A1.15|Answer

Answers (Part 1)

A1.1

Signature Σ A **signature** Σ is a set of function symbols (or operators), each associated with a specific **arity** (number of arguments).

$$\Sigma = \{(f, n) \mid f \text{ is a symbol}, n \in \mathbb{N}\}$$

It is often partitioned as $\Sigma = \bigcup_n \Sigma_n$, where Σ_n contains operators of arity n. * $c \in \Sigma_0$: Constants. * $f \in \Sigma_1$: Unary operators, etc. Part 1: Logic, Unification & Induction|Back to Q

A1.2

Terms ($T_\Sigma(X)$) The set of **terms** $T_\Sigma(X)$ over a signature Σ and a countable set of variables X is defined inductively: 1. **Variables:** If $x \in X$, then x is a term. 2. **Operators:** If $f \in \Sigma_n$ and $t_1, \ldots, t_n$ are terms, then $f(t_1, \ldots, t_n)$ is a term. *Note:* A term with no variables ($T_\Sigma(\emptyset)$) is called a **ground term** (or closed term). Part 1: Logic, Unification & Induction|Back to Q

A1.3

Substitution A substitution σ is a partial function from variables to terms:

$$\sigma : X \to T_\Sigma(X)$$

such that $\sigma(x) \neq x$ for only a finite set of variables (the *domain* of σ). It is usually denoted as $\sigma = [t_1/x_1, \ldots, t_n/x_n]$. The application of σ to a term t (written $t\sigma$) replaces simultaneously every occurrence of x_i in t with t_i. Part 1: Logic, Unification & Induction|Back to Q

A1.4

Instantiation Order Given two substitutions θ and σ, we say that θ is **more general** than σ (written $\theta \preceq \sigma$ or σ is an instance of θ) if there exists a substitution ρ such that:

$$\sigma = \rho \circ \theta$$

This means σ can be obtained by further instantiating the results of θ. Part 1: Logic, Unification & Induction|Back to Q

A1.5

Unification Problem Given a set of equations between terms $E = \{t_1 = u_1, \ldots, t_n = u_n\}$, the **unification problem** asks to find a substitution σ (called a **unifier**) that makes both sides of each equation syntactically identical:

$$\forall i.\ t_i\sigma \equiv u_i\sigma$$

Part 1: Logic, Unification & Induction|Back to Q

A1.6

Most General Unifier (MGU) A unifier μ for a problem E is a **Most General Unifier** if it is more general than any other unifier of E. Formally: $\forall\sigma$ unifier of E, $\exists\rho$ such that $\sigma = \rho \circ \mu$. *Property:* If an MGU exists, it is unique up to variable renaming. Part 1: Logic, Unification & Induction|Back to Q

A1.7

Unification Algorithm (Martelli-Montanari) The algorithm transforms a set of equations E by non-deterministically applying the following rules until termination or failure: 1. **Delete:** $\{t = t\} \cup E' \implies E'$ (Remove trivial identities). 2. **Decompose:** $\{f(t_1, \ldots, t_n) = f(u_1, \ldots, u_n)\} \cup E' \implies \{t_1 = u_1, \ldots, t_n = u_n\} \cup E'$. 3. **Conflict:** $\{f(\ldots) = g(\ldots)\} \cup E' \implies$ FAIL if $f \neq g$. 4. **Swap:** $\{t = x\} \cup E' \implies \{x = t\} \cup E'$ if t is not a variable. 5. **Eliminate:** $\{x = t\} \cup E' \implies \{x = t\} \cup E'[t/x]$ if $x \in Vars(E')$ and $x \notin Vars(t)$. 6. **Occurs Check:** $\{x = t\} \cup E' \implies$ FAIL if $x \neq t$ but $x \in Vars(t)$ (circularity, e.g., $x = f(x)$).

If it terminates with $\{x_1 = t_1, \ldots, x_k = t_k\}$, this set represents the MGU. Part 1: Logic, Unification & Induction|Back to Q

A1.8

Logical System A logical system R is defined by a set of **inference rules**. Each rule has the form:

$$\frac{P_1 \quad P_2 \quad \ldots \quad P_n}{C}$$

Where P_i are the **premises** and C is the **conclusion**. * If $n = 0$, the rule is an **axiom** (e.g., $\langle \textbf{skip}, \sigma \rangle \to \sigma$). * Formulas derivable using these rules are called **theorems** of the system. Part 1: Logic, Unification & Induction|Back to Q

A1.9

Derivations * A **derivation** (or proof) for a formula ϕ is a finite tree where: * The root is labeled by ϕ. * The leaves are instances of axioms. * Each internal node is the result of applying an inference rule to its children. Part 1: Logic, Unification & Induction|Back to Q

A1.10

Goal-Oriented Derivation A "bottom-up" strategy to find a proof. Starting from the desired conclusion (**Goal**), one applies rules backward to generate sub-goals, repeating until all sub-goals are axioms (empty set of goals). This is the basis of Logic Programming (e.g., Prolog). Part 1: Logic, Unification & Induction|Back to Q

A1.11

Well-Founded Relation A binary relation $\prec$ on a set A is **well-founded** if it admits no infinite descending chains:

$$\cdots \prec a_2 \prec a_1 \prec a_0$$

Equivalent (Minimal Element Principle): Every non-empty subset $Q \subseteq A$ has a **minimal element** m (an element such that no $x \in Q$ satisfies $x \prec m$). Part 1: Logic, Unification & Induction|Back to Q

A1.12

Principle of Well-Founded Induction Let $\prec$ be a well-founded relation on A. To prove a property $P(x)$ for all $x \in A$, it is sufficient to prove:

$$\forall x \in A.\ (\forall y \prec x.\ P(y)) \implies P(x)$$

Intuition: If $P(x)$ holds assuming it holds for all "smaller" elements, then it holds for everyone. Part 1: Logic, Unification & Induction|Back to Q

A1.13

Structural Induction A special case of Well-Founded Induction where $\prec$ is the "immediate sub-term" relation. To prove $P(t)$ for all terms $t \in T_\Sigma$: 1. Prove P for constants and variables (Base cases). 2. Prove $P(f(t_1, \ldots, t_n))$ assuming $P(t_1), \ldots, P(t_n)$ (Inductive step). *Limitation:* It is strictly tied to syntax. It works for properties of the program structure, but often fails for behavioral properties (e.g., transition semantics) where **Rule Induction** is needed. Part 1: Logic, Unification & Induction|Back to Q

A1.14

Rule Induction The fundamental principle for proving properties of systems defined by inference rules (like Operational Semantics). Let I_R be the set of theorems (derivable formulas). To prove that a property P holds for all $\delta \in I_R$: * Prove that P is **closed under the rules**. * For every rule $\frac{y_1 \ldots y_n}{x}$:

$$P(y_1) \wedge \cdots \wedge P(y_n) \implies P(x)$$

If this implication holds for all rules (including axioms, where premises are empty), then P holds for all derivable theorems. Part 1: Logic, Unification & Induction|Back to Q

A1.15

Application of Rule Induction Used to prove properties of the transition relation $\rightarrow$ (e.g., Determinacy). Since the relation $\langle c, \sigma \rangle \rightarrow \sigma'$ is defined by rules: 1. **Base Cases:** Prove the property holds for axioms (e.g., `skip`, assignment). 2. **Inductive Steps:** For rules with premises (e.g., sequence, if, while), assume the property holds for the transitions in the premises (Inductive Hypothesis) and prove it holds for the conclusion. Part 1: Logic, Unification & Induction|Back to Q

Part 2: IMP & Operational Semantics

Questions

16. **Syntax:** What is the syntax of IMP (Aexp, Bexp, Com)? A2.1|Answer
17. **Small-Step Config:** What is the configuration of a small-step semantics? A2.2|Answer
18. **Small-Step Rules:** What are the rules of the Small-Step Operational Semantics (SOS) of IMP? A2.3|Answer
19. **Big-Step Rules:** What are the rules of the Big-Step Operational Semantics of IMP? A2.4|Answer
20. **Comparison:** What is the difference between small-step and big-step semantics? A2.5|Answer
21. **Termination (Aexp):** How to prove termination of arithmetic expressions? A2.6|Answer
22. **Determinacy (Aexp):** How to prove determinacy of arithmetic expressions? A2.7|Answer
23. **Determinacy (Com):** How to prove determinacy of commands? (Crucial Proof) A2.8|Answer
24. **Divergence:** Which rule allows proving the divergence of a command? A2.9|Answer
25. **Equivalence:** What is the definition of Program Equivalence based on operational semantics? A2.10|Answer

Answers (Part 2)

A2.1

Syntax of IMP IMP is defined by three syntactic categories: 1. **Aexp** (Arithmetic Expressions): $a ::= n \mid x \mid a_0 + a_1 \mid a_0 - a_1 \ldots$ 2. **Bexp** (Boolean Expressions): $b ::= \text{true} \mid \text{false} \mid a_0 = a_1 \mid a_0 \leq a_1 \mid \neg b \mid b_0 \wedge b_1 \ldots$ 3. **Com** (Commands):

$$c ::= \textbf{skip} \mid x := a \mid c_0; c_1 \mid \textbf{if } b \textbf{ then } c_0 \textbf{ else } c_1 \mid \textbf{while } b \textbf{ do } c$$

Note: The state (memory) is a function $\sigma : \text{Loc} \to \mathbb{Z}$. Part 2: IMP & Operational Semantics|Back to Q

A2.2

Small-Step Configuration A configuration is a pair $\langle c, \sigma \rangle$ representing the command yet to be executed and the current memory. * **Transitions:** $\langle c, \sigma \rangle \to \langle c', \sigma' \rangle$ (intermediate step) or $\langle c, \sigma \rangle \to \sigma'$ (termination). * **Property:** SOS describes the computation *step-by-step*, allowing the observation of intermediate states (useful for concurrency). Part 2: IMP & Operational Semantics|Back to Q

A2.3

Small-Step Rules (Key Examples) * **Assignment:** $\langle x := n, \sigma \rangle \to \sigma[n/x]$ (where n is a value). * **Sequence:**

$$\frac{\langle c_0, \sigma \rangle \to \langle c_0', \sigma' \rangle}{\langle c_0; c_1, \sigma \rangle \to \langle c_0'; c_1, \sigma' \rangle} \quad \text{and} \quad \frac{\langle c_0, \sigma \rangle \to \sigma'}{\langle c_0; c_1, \sigma \rangle \to \langle c_1, \sigma' \rangle}$$

* **While (Unrolling):**

$$\langle \textbf{while } b \textbf{ do } c, \sigma \rangle \to \langle \textbf{if } b \textbf{ then } (c; \textbf{while } b \textbf{ do } c) \textbf{ else skip}, \sigma \rangle$$

Part 2: IMP & Operational Semantics|Back to Q

A2.4

Big-Step Rules (Natural Semantics) Describes the relation between initial state and final state directly: $\langle c, \sigma \rangle \to \sigma'$. * **Skip:** $\langle \textbf{skip}, \sigma \rangle \to \sigma$. * **Sequence:**

$$\frac{\langle c_0, \sigma \rangle \to \sigma'' \quad \langle c_1, \sigma'' \rangle \to \sigma'}{\langle c_0; c_1, \sigma \rangle \to \sigma'}$$

* **While (True):**

$$\frac{\langle b, \sigma \rangle \to \text{true} \quad \langle c, \sigma \rangle \to \sigma'' \quad \langle \textbf{while } b \textbf{ do } c, \sigma'' \rangle \to \sigma'}{\langle \textbf{while } b \textbf{ do } c, \sigma \rangle \to \sigma'}$$

Part 2: IMP & Operational Semantics|Back to Q

A2.5

Small-step vs Big-step * **Small-step (SOS):** Closer to the physical execution (abstract machine). Can distinguish between deadlock and infinite loop. Necessary for **concurrency** (interleaving). * **Big-step:** More abstract. Concise for proving properties like equivalence or writing interpreters/compilers. Cannot easily model non-terminating computations (they just have no derivation tree). Part 2: IMP & Operational Semantics|Back to Q

A2.6

Termination of Aexp We prove that "Every arithmetic expression a evaluates to a value n". **Method:** Structural Induction on a. 1. **Base Cases:** n evaluates to n; x evaluates to $\sigma(x)$. 2. **Inductive Step ($a_0 + a_1$):** Assume $a_0 \to n_0$ and $a_1 \to n_1$ (IH). Then by the rule for $+$, $a_0 + a_1 \to n_{sum}$. *Conclusion:* Expressions in IMP always terminate (no recursion/loops inside expressions). Part 2: IMP & Operational Semantics|Back to Q

A2.7

Determinacy of Aexp Theorem: $\forall a, \sigma, n, m.\ (\langle a, \sigma\rangle \to n \wedge \langle a, \sigma\rangle \to m) \implies n = m$. **Method:** Structural Induction on a (since rules for expressions are syntax-directed). * Since for every syntactic form (e.g., $a_0 + a_1$) there is exactly one applicable rule, the result is unique. Part 2: IMP & Operational Semantics|Back to Q

A2.8

Determinacy of Commands Theorem: $\forall c, \sigma, \sigma_1, \sigma_2.\ (\langle c, \sigma\rangle \to \sigma_1 \wedge \langle c, \sigma\rangle \to \sigma_2) \implies \sigma_1 = \sigma_2$. **Method: Rule Induction** on the derivation of $\langle c, \sigma\rangle \to \sigma_1$. *(Structural induction fails because of the* `while` *loop).*

Proof Sketch: Let $P(\langle c, \sigma\rangle \to \sigma_1)$ be the property: "for all σ_2, if $\langle c, \sigma\rangle \to \sigma_2$ then $\sigma_1 = \sigma_2$". 1. **Base Cases (Skip/Assign):** Trivial (only one rule applies). 2. **Step (Sequence $c_0; c_1$):** * Premises: $\langle c_0, \sigma\rangle \to \sigma'$ and $\langle c_1, \sigma'\rangle \to \sigma_1$. * IH: Determinacy holds for c_0 transition and c_1 transition. * Assume another derivation $\langle c_0; c_1, \sigma\rangle \to \sigma_2$. This *must* come from $\langle c_0, \sigma\rangle \to \sigma''$ and $\langle c_1, \sigma''\rangle \to \sigma_2$. * By IH on c_0, $\sigma' = \sigma''$. * By IH on c_1 (starting from same σ'), $\sigma_1 = \sigma_2$. QED. Part 2: IMP & Operational Semantics|Back to Q

A2.9

Divergence A command c **diverges** in σ if there is no state σ' such that $\langle c, \sigma\rangle \to \sigma'$ (in Big-Step). In Small-Step, it corresponds to an **infinite sequence** of transitions:

$$\langle c, \sigma\rangle \to \langle c_1, \sigma_1\rangle \to \langle c_2, \sigma_2\rangle \to \dots$$

Proof: To prove divergence formally, we usually find a property (invariant) preserved by the small-step rules that prevents reaching a terminal state. Part 2: IMP & Operational Semantics|Back to Q

A2.10

Operational Equivalence Two commands c_1, c_2 are operationally equivalent ($c_1 \sim_{op} c_2$) if, for all starting states σ: 1. They both diverge; OR 2. They both terminate in the same state σ'. Formally (using Big-Step):

$$\forall \sigma, \sigma'.\ (\langle c_1, \sigma\rangle \to \sigma' \iff \langle c_2, \sigma\rangle \to \sigma')$$

Part 2: IMP & Operational Semantics|Back to Q

Part 3: Orders, Fixpoints & Domain Theory

Questions

26. **Partial Order:** What is a Partial Order (PO)? A3.1|Answer
27. **CPO:** What is a Complete Partial Order (CPO) and a Chain? A3.2|Answer

28. **Bottom:** What is a pointed CPO ($CPO_\perp$)? A3.3|Answer
29. **Monotonicity:** What is a monotone function? A3.4|Answer
30. **Continuity:** What is a continuous function? A3.5|Answer
31. **Kleene's Theorem: What is Kleene's Fixpoint Theorem?** (Statement & Proof Sketch) A3.6|Answer
32. **Knaster-Tarski:** What is the Knaster-Tarski Theorem? A3.7|Answer
33. **ICO: What is the Immediate Consequence Operator $\hat{R}$?** A3.8|Answer
34. **Logic Fixpoint:** How can the set of theorems of a logical system be seen as a fixpoint? A3.9|Answer
35. **Discrete/Flat:** What is a discrete domain and a flat domain? A3.10|Answer
36. **Constructions:** How are Product ($D \times E$) and Function Space ($[D \to E]$) domains defined? A3.11|Answer
37. **Lifting:** What is the Lifted Domain $D_\perp$? A3.12|Answer

Answers (Part 3)

A3.1

Partial Order (PO) A partial order is a pair $(D, \sqsubseteq)$ where D is a set and $\sqsubseteq$ is a binary relation that is: 1. **Reflexive:** $\forall d \in D.\ d \sqsubseteq d$. 2. **Transitive:** $\forall d_1, d_2, d_3.\ d_1 \sqsubseteq d_2 \wedge d_2 \sqsubseteq d_3 \implies d_1 \sqsubseteq d_3$. 3. **Antisymmetric:** $\forall d_1, d_2.\ d_1 \sqsubseteq d_2 \wedge d_2 \sqsubseteq d_1 \implies d_1 = d_2$. Part 3: Orders, Fixpoints & Domain Theory|Back to Q

A3.2

Chain & CPO * **Chain:** A sequence $\{d_i\}_{i \in \mathbb{N}}$ of elements in D such that $\forall i.\ d_i \sqsubseteq d_{i+1}$ (increasing sequence). * **CPO (Complete Partial Order):** A partial order $(D, \sqsubseteq)$ is complete if every chain $\{d_i\}$ has a **Least Upper Bound (LUB)**, denoted as $\bigsqcup_{i \in \mathbb{N}} d_i$, in D. Part 3: Orders, Fixpoints & Domain Theory|Back to Q

A3.3

Pointed CPO ($CPO_\perp$) A CPO is **pointed** (or *with bottom*) if it contains a least element, denoted $\perp$ (bottom), such that $\forall d \in D.\ \perp \sqsubseteq d$. *Relevance:* $\perp$ represents "no information", "non-termination", or "undefined". Part 3: Orders, Fixpoints & Domain Theory|Back to Q

A3.4

Monotone Function A function $f : D \to E$ between POs is **monotone** if it preserves the order:

$$\forall d_1, d_2 \in D.\ d_1 \sqsubseteq_D d_2 \implies f(d_1) \sqsubseteq_E f(d_2)$$

Intuition: More information in input yields at least as much information in output. Part 3: Orders, Fixpoints & Domain Theory|Back to Q

A3.5

Continuous Function A function $f : D \to E$ between CPOs is **continuous** if it preserves limits of chains: For every chain $\{d_i\}_{i \in \mathbb{N}}$ in D:

$$f\left(\bigsqcup_{i \in \mathbb{N}} d_i\right) = \bigsqcup_{i \in \mathbb{N}} f(d_i)$$

Note: Continuity implies monotonicity. *Relevance:* Allows computing the result of a limit by computing the limit of the results. Part 3: Orders, Fixpoints & Domain Theory|Back to Q

A3.6

Kleene's Fixpoint Theorem Statement: Let $(D, \sqsubseteq)$ be a pointed CPO ($CPO_\perp$) and $f : D \to D$ be a **continuous** function. Then f has a **least fixpoint** (lfp), computed as:

$$\text{fix}(f) = \bigsqcup_{n \in \mathbb{N}} f^n(\perp)$$

where $f^0(\perp) = \perp$ and $f^{n+1}(\perp) = f(f^n(\perp))$.

Proof Sketch (Required in exam): 1. **Chain Construction:** Since $\perp \sqsubseteq f(\perp)$ and f is monotone, by induction $\perp \sqsubseteq f(\perp) \sqsubseteq f^2(\perp) \ldots$ is a chain. 2. **Existence:** Since D is a CPO, the limit $u = \bigsqcup f^n(\perp)$ exists. 3. **Fixpoint Property:** $f(u) = f(\bigsqcup f^n(\perp)) = (\text{continuity}) = \bigsqcup f(f^n(\perp)) = \bigsqcup f^{n+1}(\perp) = u$. 4. **Least Property:** Let d be another fixpoint ($f(d) = d$). Since $\perp \sqsubseteq d$, by monotonicity $f^n(\perp) \sqsubseteq f^n(d) = d$. Thus $u = \bigsqcup f^n(\perp) \sqsubseteq d$. Part 3: Orders, Fixpoints & Domain Theory|Back to Q

A3.7

Knaster-Tarski Theorem Let $(L, \sqsubseteq)$ be a **Complete Lattice** (limits exist for *any* subset, not just chains) and $f : L \to L$ be a **monotone** function. Then the set of fixpoints of f is a complete lattice itself, and:

$$\text{lfp}(f) = \sqcap\{x \in L \mid f(x) \sqsubseteq x\}$$

(Used when we don't have continuity, e.g., for sets of derivation rules, but Kleene is preferred for computability). Part 3: Orders, Fixpoints & Domain Theory|Back to Q

A3.8

Immediate Consequence Operator ($\hat{R}$) Given a logical system with rules R, $\hat{R} : \wp(\text{Formulas}) \to \wp(\text{Formulas})$ is defined as:

$$\hat{R}(S) = \{y \mid \exists \frac{x_1 \ldots x_n}{y} \in R \text{ s.t. } \{x_1, \ldots, x_n\} \subseteq S\}$$

It calculates all facts that can be derived in *exactly one step* from the set of known facts S. *Prop:* $\hat{R}$ is continuous on the powerset lattice. Part 3: Orders, Fixpoints & Domain Theory|Back to Q

A3.9

Fixpoint Semantics of Logic The set of all provable theorems I_R is the **least fixpoint** of the operator $\hat{R}$.

$$I_R = \bigcup_{n \in \mathbb{N}} \hat{R}^n(\emptyset)$$

This relates logical derivation (proof trees) with domain theory (limits of approximation). Part 3: Orders, Fixpoints & Domain Theory|Back to Q

A3.10

Discrete & Flat Domains * **Discrete Domain:** A set ordered by identity only ($d_1 \sqsubseteq d_2 \iff d_1 = d_2$). No approximation, only equality. * **Flat Domain ($S_\perp$):** A discrete domain S augmented with a bottom $\perp$. Order: $\perp \sqsubseteq x$ for all x, and $x \sqsubseteq y \iff x = y$ for $x, y \neq \perp$. (Used for integers, booleans in IMP). Part 3: Orders, Fixpoints & Domain Theory|Back to Q

A3.11

Product and Function Domains * **Product ($D \times E$):** Pairs (d, e). Order is component-wise:

$$(d_1, e_1) \sqsubseteq (d_2, e_2) \iff d_1 \sqsubseteq_D d_2 \wedge e_1 \sqsubseteq_E e_2$$

Limit is component-wise. * **Function Space ($[D \to E]$):** The set of **continuous** functions from D to E. Order is point-wise:

$$f \sqsubseteq g \iff \forall d \in D.\ f(d) \sqsubseteq_E g(d)$$

Limit is point-wise: $(\bigsqcup f_i)(d) = \bigsqcup(f_i(d))$. Part 3: Orders, Fixpoints & Domain Theory|Back to Q

A3.12

Lifted Domain ($D_\perp$) Used to distinguish "undefined" from "defined but bottom" (in lazy evaluation) or simply to add a bottom to a domain. Elements: $D \cup \{\perp_{new}\}$. Order: $\perp_{new} \sqsubseteq x$ for all x, and elements of D keep their original order. *(Critical for HOFL Denotational Semantics).* Part 3: Orders, Fixpoints & Domain Theory|Back to Q

Part 4: HOFL & Denotational Semantics

Questions

38. **Syntax:** What is the syntax of HOFL? What is the difference between pre-terms and terms? A4.1|Answer
39. **Typing:** What are the rules of the Type System (Judgments $\Gamma \vdash t : \tau$)? A4.2|Answer
40. **Principal Type:** What is a Principal Type? A4.3|Answer
41. **Canonical Forms:** What is a Canonical Form in HOFL? A4.4|Answer
42. **Operational Semantics:** What are the rules for the Lazy Operational Semantics of HOFL? A4.5|Answer
43. **Evaluation Strategy:** What is the difference between Lazy (Call-by-Name) and Eager (Call-by-Value)? A4.6|Answer
44. **Domains:** How are the semantic domains for types (D_τ) defined? A4.7|Answer
45. **Denotational Semantics:** How is the interpretation function $[\![t]\!]_\rho$ defined? A4.8|Answer
46. **Substitution Lemma: What is the Substitution Lemma?** (Crucial for consistency) A4.9|Answer
47. **Consistency:** What is the relation between Operational and Denotational semantics? A4.10|Answer

Answers (Part 4)

A4.1

HOFL Syntax * **Pre-terms:** The raw grammar of the language:

$$t ::= x \mid n \mid t_0 + t_1 \mid \textbf{if } t \textbf{ then } t_0 \textbf{ else } t_1 \mid (t_0, t_1) \mid \textbf{fst}(t) \mid \textbf{snd}(t) \mid \lambda x.t \mid t_0 t_1 \mid \textbf{rec } x.t$$

* **Terms:** The subset of pre-terms that are **well-typed** according to the type system. Only terms have a guaranteed semantics. Part 4: HOFL & Denotational Semantics|Back to Q

A4.2

Type System Rules Judgments of form $\Gamma \vdash t : \tau$, where Γ assigns types to free variables. * **Var:** $\Gamma, x : \tau \vdash x : \tau$ * **Abs:** $\frac{\Gamma, x:\tau_1 \vdash t:\tau_2}{\Gamma \vdash \lambda x.t:\tau_1 \to \tau_2}$ * **App:** $\frac{\Gamma \vdash t_1:\tau_1 \to \tau_2 \quad \Gamma \vdash t_2:\tau_1}{\Gamma \vdash t_1 t_2:\tau_2}$ * **Rec:** $\frac{\Gamma, x:\tau \vdash t:\tau}{\Gamma \vdash \textbf{rec } x.t:\tau}$ (Fixed point requires domain consistency). Part 4: HOFL & Denotational Semantics|Back to Q

A4.3

Principal Type A type τ is the **principal type** of a term t if: 1. $\Gamma \vdash t : \tau$ is derivable. 2. For any other derivable type τ' for t, there exists a substitution σ such that $\tau' = \tau\sigma$. *Algorithm:* Uses Unification (Robinson/Martelli-Montanari) to infer the most general type constraints. Part 4: HOFL & Denotational Semantics|Back to Q

A4.4

Canonical Forms The values that result from a terminating computation.

$$c ::= n \mid (t_0, t_1) \mid \lambda x.t$$

Note: In Lazy semantics, the components of a pair (t_0, t_1) are *not* required to be canonical forms themselves. Part 4: HOFL & Denotational Semantics|Back to Q

A4.5

Lazy Operational Semantics ($t \downarrow c$) Big-step semantics where arguments are not evaluated before function application. * **App:**

$$\frac{t_1 \downarrow \lambda x.t_1' \quad t_1'[t_2/x] \downarrow c}{t_1 t_2 \downarrow c}$$

(Note that t_2 is substituted `as is` *into the body).* * **Fst:**

$$\frac{t \downarrow (t_1, t_2) \quad t_1 \downarrow c}{\mathbf{fst}(t) \downarrow c}$$

Part 4: HOFL & Denotational Semantics|Back to Q

A4.6

Lazy vs Eager * **Lazy (Call-by-Name):** Arguments are substituted unevaluated. Functions are applied immediately. * *Advantage:* Can handle infinite structures (streams). Terminates more often. * **Eager (Call-by-Value):** Arguments are evaluated to canonical form *before* application. * Rule: $\frac{t_1 \downarrow \lambda x.t_1' \quad t_2 \downarrow c_2 \quad t_1'[c_2/x] \downarrow c}{t_1 t_2 \downarrow c}$ Part 4: HOFL & Denotational Semantics|Back to Q

A4.7

Semantic Domains Defined inductively on types to handle partiality (lifting). * $D_{int} = \mathbb{Z}_\perp$ (Flat domain of integers). * $D_{\tau_1 \times \tau_2} = (D_{\tau_1} \times D_{\tau_2})_\perp$ (Lifted product). * $D_{\tau_1 \to \tau_2} = [D_{\tau_1} \to D_{\tau_2}]_\perp$ (Lifted function space). * *Why Lifted?* To distinguish between a diverging computation ($\perp$) and a computation that returns a value (even if that value is a function or pair). Part 4: HOFL & Denotational Semantics|Back to Q

A4.8

Denotational Semantics ($[\![t]\!]_\rho$) Maps a term t and an environment ρ to an element in D_τ. * $[\![n]\!]_\rho = \lfloor n \rfloor$ (Lifted value). * $[\![\lambda x.t]\!]_\rho = \lfloor \boldsymbol{\lambda} d.[\![t]\!]_{\rho[d/x]} \rfloor$ (Returns a lifted continuous function). * $[\![t_1 t_2]\!]\rho =$ Let $\varphi \Leftarrow [\![t_1]\!]\rho$ in $\varphi([\![t_2]\!]\rho)$ (Monadic bind: if t_1 diverges, result is $\perp$). * $[\![\mathbf{rec}\ x.t]\!]_\rho = \mathrm{fix}(\boldsymbol{\lambda} d.[\![t]\!]_{\rho[d/x]})$. Part 4: HOFL & Denotational Semantics|Back to Q

A4.9

Substitution Lemma The denotation of a term with a substitution is equal to the denotation of the term in an updated environment.

$$[\![t[t'/x]]\!]_\rho = [\![t]\!]_{\rho[[\![t']\!]_\rho/x]}$$

Importance: It is the key step to prove that the operational rule for application (which uses substitution) matches the denotational definition (which uses environment update). Part 4: HOFL & Denotational Semantics|Back to Q

A4.10

Consistency Theorem: For any closed term t and canonical form c:

$$t \downarrow c \implies [\![t]\!] = [\![c]\!]$$

Interpretation: The Operational semantics is **correct** with respect to Denotational semantics. *Note:* The converse (Completeness) does NOT hold in the standard model (due to the "Parallel OR" problem, although for HOFL/PC it's more about full abstraction issues). Part 4: HOFL & Denotational Semantics|Back to Q

Part 5: Concurrency (CCS)

Questions

48. **Syntax:** What is the syntax of CCS? A5.1|Answer
49. **LTS:** What is the Labeled Transition System (LTS) for CCS? A5.2|Answer
50. **Derivation:** How is the `Com` (Communication) rule defined? A5.3|Answer
51. **Strong Bisimulation:** What is the definition of Strong Bisimulation? A5.4|Answer
52. **Bisimilarity:** What is Strong Bisimilarity ($\sim$)? A5.5|Answer
53. **Game:** What are the rules of the Bisimulation Game? A5.6|Answer
54. **Congruence:** Is Strong Bisimilarity a congruence? A5.7|Answer
55. **Verification:** How to prove that two processes are strongly bisimilar? A5.8|Answer

Answers (Part 5)

A5.1

CCS Syntax

$$P ::= \mathbf{nil} \mid \mu.P \mid P \setminus L \mid P[f] \mid P_1 + P_2 \mid P_1 \mid P_2 \mid K$$

* μ: Action (α input, $\bar{\alpha}$ output, or τ internal). * $\setminus L$: Restriction (hides actions in set L). * $[f]$: Relabeling. * $+$: Non-deterministic choice. * $|$: Parallel composition. Part 5: Concurrency (CCS)|Back to Q

A5.2

LTS Rules Define transitions $P \xrightarrow{\mu} P'$. * **Act:** $\mu.P \xrightarrow{\mu} P$ * **Sum:** If $P_1 \xrightarrow{\mu} P_1'$, then $P_1 + P_2 \xrightarrow{\mu} P_1'$ (Choice is resolved). * **Par:** Processes can move independently ($P_1 \mid P_2 \xrightarrow{\mu} P_1' \mid P_2$) or synchronize. Part 5: Concurrency (CCS)|Back to Q

A5.3

Communication Rule (Com) Describes the synchronization (handshake) between two parallel processes.

$$\frac{P_1 \xrightarrow{\alpha} P_1' \quad P_2 \xrightarrow{\bar{\alpha}} P_2'}{P_1 \mid P_2 \xrightarrow{\tau} P_1' \mid P_2'}$$

Result: An internal silent action τ. Part 5: Concurrency (CCS)|Back to Q

A5.4

Strong Bisimulation (R) A binary relation R is a strong bisimulation if for all $(P, Q) \in R$: 1. If $P \xrightarrow{\mu} P'$, then $\exists Q'$ such that $Q \xrightarrow{\mu} Q'$ and $(P', Q') \in R$. 2. If $Q \xrightarrow{\mu} Q'$, then $\exists P'$ such that $P \xrightarrow{\mu} P'$ and $(P', Q') \in R$. *Slogan:* "Every move of one must be matched by the other to reach equivalent states." Part 5: Concurrency (CCS)|Back to Q

A5.5

Strong Bisimilarity ($\sim$) It is the union of all strong bisimulations (the largest bisimulation).

$$P \sim Q \iff \exists R \text{ bisimulation s.t. } (P, Q) \in R$$

It is an **equivalence relation**. Part 5: Concurrency (CCS)|Back to Q

A5.6

Bisimulation Game Played by **Attacker** and **Defender** on pair (P, Q). 1. Attacker chooses one process (say P) and makes a move $P \xrightarrow{\mu} P'$. 2. Defender must match with the other process $Q \xrightarrow{\mu} Q'$. 3. Game continues from (P', Q'). *Result:* If Attacker gets stuck (cannot move), Defender wins. If Attacker makes a move Defender cannot match, Attacker wins ($P \not\sim Q$). Infinite game = Defender wins ($P \sim Q$). Part 5: Concurrency (CCS)|Back to Q

A5.7

Congruence Strong bisimilarity $\sim$ **IS** a congruence with respect to all CCS operators. *Meaning:* If $P \sim Q$, then $C[P] \sim C[Q]$ for any context C. *(Note: This allows substituting equivalent components in modular design).* Part 5: Concurrency (CCS)|Back to Q

A5.8

Proving Bisimilarity To prove $P \sim Q$: 1. **Exhibit a relation** $R = \{(P, Q), \dots\}$. 2. Show that R is a bisimulation (Check moves for every pair in R). * (Use diagrams to map states) 3. Alternatively, use algebraic laws (e.g., $P + P = P$, $P \mid \mathbf{nil} = P$). Part 5: Concurrency (CCS)|Back to Q

Exercises

This file aggregates exercises from all course modules. It covers Domain Theory, Semantics, HOFL, Concurrency, and Real Languages.

---

1. Domain Theory & Logic

Ex 1.1: Continuity on Power Sets

Topic: CPO, Continuity. Consider the CPO $(\wp(\mathbb{N}), \subseteq)$. Let $S \subseteq \mathbb{N}$ be a fixed set. Prove that the function $f_S : \wp(\mathbb{N}) \to \wp(\mathbb{N})$ defined as $f_S(X) = X \cap S$ is **continuous**.

Sol 1.1|Go to Solution

Ex 1.2: Divisors CPO

Topic: Partial Orders, Bottom. Let $D = \{n \in \mathbb{N} \mid n > 0\} \cup \{\infty\}$. Define a relation $\sqsubseteq$ such that: * For $n, m \in \mathbb{N}$, $n \sqsubseteq m$ iff n divides m. * For any $x \in D$, $x \sqsubseteq \infty$.

1. Is $(D, \sqsubseteq)$ a CPO?
2. Does it have a bottom element?

Sol 1.2|Go to Solution

Ex 1.3: Composition of Continuous Functions

Topic: Continuity, Fixpoints. Let D, E be $CPO_\perp$ and $f : D \to E, g : E \to D$ be continuous functions. Let $h = g \circ f$ and $k = f \circ g$. Let $e_0 = \text{fix}(k)$. Prove that $g(e_0)$ is a fixpoint for h.

Sol 1.3|Go to Solution

---

2. IMP Semantics

Ex 2.1: Operational Semantics of for loops

Topic: Language Extension, Big-Step rules. Replace the `while` command in IMP with a `for a do c` command. The loop executes c exactly N times, where N is the value of a in the initial state. 1. Define the Big-Step inference rules for `for`. 2. Prove that the command always terminates (assuming c terminates).

Sol 2.1|Go to Solution

Ex 2.2: Divergence of while

Topic: Denotational Semantics, Fixpoints. Consider the command $w = \textbf{while } x > 0 \textbf{ do } x := x + 1$. Using the denotational semantics (Fixpoint of Γ), prove that $\mathcal{C}[\![w]\!]\sigma = \perp$ for any σ where $\sigma(x) > 0$.

Sol 2.2|Go to Solution

---

3. HOFL & Haskell

Ex 3.1: Typing Derivation

Topic: HOFL Type System. Derive the type for the term $t = \lambda f.\lambda x.(f(fx))$.

Sol 3.1|Go to Solution

Ex 3.2: Haskell - Palindromes

Topic: Functional Programming, Lists. Write a Haskell function `pal` that checks if a list is a palindrome. Then write `pals` that filters a list of strings, keeping only palindromes.

Sol 3.2|Go to Solution

Ex 3.3: Haskell - Quicksort

Topic: Recursion, List Comprehension. Implement the Quicksort algorithm in Haskell using list comprehensions.

Sol 3.3|Go to Solution

---

4. Concurrency (CCS)

Ex 4.1: Guardedness

Topic: CCS Syntax. Check if the following processes are **guarded**: 1. $A \stackrel{\text{def}}{=} \alpha.A + \beta.\textbf{nil}$ 2. $B \stackrel{\text{def}}{=} \text{rec } x.(x \mid \alpha.\textbf{nil})$ 3. $C \stackrel{\text{def}}{=} \text{rec } x.(\alpha.x \mid \beta.x)$

Sol 4.1|Go to Solution

Ex 4.2: Strong Bisimulation Game

Topic: Strong Bisimulation. Let $P = a.(b.\mathbf{nil} + c.\mathbf{nil})$ and $Q = a.b.\mathbf{nil} + a.c.\mathbf{nil}$. Show that $P \not\sim Q$ by describing the winning strategy for the Attacker (Alice).

Sol 4.2|Go to Solution

Ex 4.3: Weak Bisimulation (Tau-laws)

Topic: Weak Bisimulation. Prove that $P + \tau.P \approx \tau.P$. (Hint: Construct a weak bisimulation relation $\mathcal{R}$ containing the pair).

Sol 4.3|Go to Solution

Ex 4.4: Buffer Capacity

Topic: Modeling. Show that a buffer of capacity 2 implemented as two parallel cells $(B_0^1 \mid B_0^1)$ is **NOT** bisimilar to a sequential 2-position buffer. (Focus on the traces).

Sol 4.4|Go to Solution

Ex 4.5: HML Formula

Topic: Logic. Find an HML formula that distinguishes P from Q in Ex 4.2.

Sol 4.5|Go to Solution

---

5. Real Languages (Go/Erlang)

Ex 5.1: Erlang Temperature Server

Topic: Actors, Message Passing. Write an Erlang server that: 1. Receives `{Pid, celsius, C}` and replies `{self(), fahrenheit, F}`. 2. Receives `{Pid, fahrenheit, F}` and replies `{self(), celsius, C}`. 3. Handles a `stop` message.

Sol 5.1|Go to Solution

Ex 5.2: Go Pairing

Topic: Channels, Goroutines. Write a Go function `pairing(in1, in2 chan int) chan [2]int` that reads one integer from `in1`, one from `in2`, pairs them, and sends the array on the output channel.

Sol 5.2|Go to Solution

Solutions

Sol 1.1

Proof: We need to show $f_S(\bigcup_i X_i) = \bigcup_i f_S(X_i)$ for any chain $\{X_i\}$. * **LHS:** $f_S(\bigcup X_i) = (\bigcup X_i) \cap S$. * **RHS:** $\bigcup(f_S(X_i)) = \bigcup(X_i \cap S)$. * By the general distributive law of set theory $((\bigcup A_i) \cap B = \bigcup(A_i \cap B))$, LHS = RHS. * Therefore, f_S is continuous.

Ex 1.1: Continuity on Power Sets|Back to Exercise

Sol 1.2

1. **Yes, it is a CPO.** The relation is reflexive (n divides n), antisymmetric (n|m & m|n $\implies$ n=m), transitive.
   - Chains in $\mathbb{N}$ must be finite because n divides m implies $n \leq m$ (or $m = 0$, handled separately). Infinite chains must eventually stabilize or go to ∞.
   - Limit of any chain exists (max element or ∞).
2. **Bottom:** Yes, 1 divides every number. $\perp = 1$.

Ex 1.2: Divisors CPO|Back to Exercise

Sol 1.3

We need to prove $h(g(e_0)) = g(e_0)$. 1. $e_0 = \text{fix}(k)$, so $k(e_0) = e_0$. 2. Substitute definitions: $k = f \circ g$, so $f(g(e_0)) = e_0$. 3. Apply g to both sides: $g(f(g(e_0))) = g(e_0)$. 4. Since $h = g \circ f$, we have $h(g(e_0)) = g(e_0)$. Thus, $g(e_0)$ is a fixpoint of h.

Ex 1.3: Composition of Continuous Functions|Back to Exercise

Sol 2.1

Rules:

$$\frac{\langle a, \sigma\rangle \to N \quad N \leq 0}{\langle \textbf{for } a \textbf{ do } c, \sigma\rangle \to \sigma} \qquad \frac{\langle a, \sigma\rangle \to N \quad N > 0 \quad \langle c; \textbf{for } (N-1) \textbf{ do } c, \sigma\rangle \to \sigma'}{\langle \textbf{for } a \textbf{ do } c, \sigma\rangle \to \sigma'}$$

Termination: Proved by mathematical induction on the value $N = \mathcal{C}[\![a]\!]\sigma$. Base case ($N \leq 0$) terminates immediately (skip). Step (N) reduces to execution of c (terminates by hypothesis) followed by `for` on $N - 1$, which terminates by Inductive Hypothesis.

Ex 2.1: Operational Semantics of for loops|Back to Exercise

Sol 2.2

$\Gamma(\varphi)(\sigma) = \text{cond}(\sigma(x) > 0, \varphi(\sigma[x + 1/x]), \sigma)$. Let's compute approximations for a state σ with $\sigma(x) > 0$: * $\Gamma^0(\perp)\sigma = \perp$. * $\Gamma^1(\perp)\sigma = \text{cond}(\text{true}, \perp, \sigma) = \perp$. * By induction, $\Gamma^n(\perp)\sigma = \perp$ for all n. The fixpoint is the limit $\bigsqcup \perp = \perp$. The program diverges.

Ex 2.2: Divergence of while|Back to Exercise

Sol 3.1

Term: $\lambda f.\lambda x.f(fx)$. 1. Assume $x : \alpha$. 2. For $f(x)$ to be valid, $f : \alpha \to \beta$. 3. The result of $f(x)$ is type β. 4. For outer $f(\dots)$ to be valid, f takes β. So $\alpha = \beta$. 5. Thus $f : \alpha \to \alpha$. 6. Result type is α. 7. Total type: $(\alpha \to \alpha) \to \alpha \to \alpha$.

Ex 3.1: Typing Derivation|Back to Exercise

Sol 3.2

```haskell
pal :: Eq a => [a] -> Bool
pal xs = xs == reverse xs

pals :: [String] -> [String]
pals list = filter pal list
```

Ex 3.2: Haskell - Palindromes|Back to Exercise

Sol 3.3

Haskell

```
qsort :: Ord a => [a] -> [a]
qsort [] = []
qsort (x:xs) = qsort smaller ++ [x] ++ qsort larger
    where
        smaller = [a | a <- xs, a <= x]
        larger  = [b | b <- xs, b > x]
```

Ex 3.3: Haskell - Quicksort|Back to Exercise

Sol 4.1

1. **Guarded.** A appears under α.
2. **Unguarded.** x appears in parallel with $\alpha.\mathbf{nil}$, not under a prefix. (Infinite branching).
3. **Guarded.** x appears under α and β.

Ex 4.1: Guardedness|Back to Exercise

Sol 4.2

Attacker Strategy:

1. Alice plays $Q \xrightarrow{a} b.\mathbf{nil}$ (left branch).
2. Bob must play $P \xrightarrow{a} b.\mathbf{nil} + c.\mathbf{nil}$ (only move).
3. Current state: $(b.\mathbf{nil} + c.\mathbf{nil}, \quad b.\mathbf{nil})$.
4. Alice plays Left side: $\xrightarrow{c} \mathbf{nil}$.
5. Bob (on Right side $b.\mathbf{nil}$) cannot perform c. **Alice wins.**

Ex 4.2: Strong Bisimulation Game|Back to Exercise

Sol 4.3

Relation $\mathcal{R} = \{(P + \tau.P, \tau.P)\} \cup Id$.

- **Challenge $P + \tau.P$:**
  - Move $P + \tau.P \xrightarrow{\tau} P$ (right option). Defender matches $\tau.P \xRightarrow{\tau} P$. Result $(P, P) \in Id$.
  - Move $P + \tau.P \xrightarrow{\alpha} P'$ (left option). Defender matches $\tau.P \xrightarrow{\tau} P \xrightarrow{\alpha} P'$. Result $(P', P') \in Id$.
- **Challenge $\tau.P$:**
  - Move $\tau.P \xrightarrow{\tau} P$. Defender matches $P + \tau.P \xrightarrow{\tau} P$ (right option). Result $(P, P) \in Id$.

Ex 4.3: Weak Bisimulation (Tau-laws)|Back to Exercise

Sol 4.4

A sequential buffer B_2 guarantees FIFO order.

Parallel buffer $B_{par} = B_0^1 \mid B_0^1$:

1. Input a (left), Input b (right). State: $B_1^1(a) \mid B_1^1(b)$.
2. Can output b *before* a (race condition on output channels).
3. Sequential buffer cannot do this. Trace $\{in(a), in(b), out(b)\}$ is possible for Parallel but not Sequential.

Ex 4.4: Buffer Capacity|Back to Exercise

Sol 4.5

Formula: $[a](\langle b\rangle \mathrm{tt} \wedge \langle c\rangle \mathrm{tt})$.

- $P \models F$ because after a, state is $(b + c)$ which enables both b and c.
- $Q \not\models F$ because after a it reaches either $b.\mathbf{nil}$ (no c) or $c.\mathbf{nil}$ (no b). The box $[a]$ requires the property to hold for *all* outcomes.

Ex 4.5: HML Formula|Back to Exercise

Sol 5.1

Erlang

```
-module(tempserver).
-export([start/0, loop/0]).

start() -> spawn(tempserver, loop, []).

loop() ->
    receive
        {Pid, celsius, C} ->
            F = 1.8 * C + 32,
            Pid ! {self(), fahrenheit, F},
            loop();
        {Pid, fahrenheit, F} ->
            C = (F - 32) / 1.8,
            Pid ! {self(), celsius, C},
            loop();
        stop ->
            true
    end.
```

Ex 5.1: Erlang Temperature Server|Back to Exercise

Sol 5.2

Go

```
func pairing(in1, in2 chan int) chan [2]int {
    out := make(chan [2]int)
    go func() {
        for {
            v1 := <-in1
            v2 := <-in2
            out <- [2]int{v1, v2}
        }
    }()
    return out
}
```

Ex 5.2: Go Pairing|Back to Exercise